



2022 - 2023

GRADUATION PROJECT

NATIONAL ENGINEERING DEGREE

**SPECIALTY : Software Architecture
Engineering**

TITLE : MediaNet incubator Platform

By: *Yassine cherni*

Academic supervisor: *Slim Guermazi*

Corporate Internship Supervisor: *Yassine Mechich*

MEDIANET

Je valide le dépôt du rapport PFE relatif à l'étudiant nommé ci-dessous / I
validate the submission of the student's report:

- Nom & Prénom /Name & Surname : Yassine Cherni

Encadrant Entreprise/ Business site Supervisor

- Nom & Prénom /Name & Surname : Yassine Mechich

Cachet & Signature / Stamp & Signature

Encadrant Académique/Academic Supervisor

- Nom & Prénom /Name & Surname : Slim Guerhazi

Signature / Signature

Dedication

To my parents, for their unconditional support and everything they sacrificed along the way,

To my teachers, who passed on their knowledge with patience and rigor,

To my friends and colleagues, for their encouragement and their presence,

To everyone who, directly or indirectly, contributed to this work,

I dedicate this modest achievement.

— Yassine Cherni

Acknowledgements

I would first like to thank **Medianet** for welcoming me and giving me the opportunity to carry out this end-of-studies project in a genuinely stimulating professional environment.

My sincere thanks go to my company supervisor, **Yassine Mechich**, for their availability, their sound advice, and the rigorous follow-up they provided throughout this internship.

I would also like to thank my academic supervisor, **Slim Guermazi**, for their methodological guidance, their constructive feedback, and their availability despite a busy schedule.

I address my thanks to the members of the jury for the honor they do me by agreeing to evaluate this work.

Finally, I thank the entire teaching staff of **ESPRIT** for the quality of the training I received during my studies, as well as my family and friends for their constant support.

Table of Contents

Dedication	1
Acknowledgements	4
Table of Contents	5
List of Figures	7
List of Tables.....	8
General Introduction	1
Chapter 1 — General Framework	3
Introduction	3
1.1 Host Organization	3
1.1.1 General Presentation	3
1.1.2 Fields of Activity.....	3
1.1.3 Company Organization	4
1.1.4 Host Department	4
1.2 Project Overview	4
1.3 Study of the Existing Landscape	5
1.3.1 Description of Existing Practices	5
1.3.2 Critique of the Existing Landscape	6
1.4 Proposed Solution	7
1.5 Development Methodology.....	7
1.5.1 The Waterfall Cycle	8
1.5.2 Agile Methods (Scrum).....	8
1.5.3 Methodology Chosen	8
Conclusion.....	9
Chapter 3 — Requirements Analysis and Specification	14
Introduction	14
3.1 Project Vision.....	14
3.1.1 Functional Requirements.....	14
3.1.2 Non-Functional Requirements	15
3.2 Requirements Specification.....	16
3.2.1 Identification of Actors	16
3.2.2 Identification of Use Cases	17
3.3 Detailed Requirements Specification	19
3.3.1 Textual Description of Major Use Cases	19
3.3.2 System Sequence Diagrams of Major Use Cases.....	24
3.3.3 Activity Diagrams	24

Conclusion.....	26
Chapter 4 — Design	27
Introduction	27
4.1 Class Diagram	27
4.1.1 Design Class Diagram.....	27
4.1.2 Class Descriptions	29
4.1.3 Data Dictionary	30
4.2 Design Sequence Diagrams.....	32
4.2.1 Tracked Session Notification	32
4.2.2 Visibility-Based Session Filtering.....	32
4.3 Component Diagram	33
Conclusion.....	33
Chapter 5 — Implementation	35
Introduction	35
5.1 Work Environment	35
5.1.1 Software Environment.....	35
5.1.2 Hardware Environment	35
5.1.3 Frameworks	35
5.1.4 Programming Languages.....	35
5.2 Architecture Adopted	36
5.3 Interfaces Delivered	37
5.4 Project Planning	38
5.5 Difficulties Encountered and Changes Made	39
5.6 Test Dataset	41
Conclusion.....	42
General Conclusion	46
Bibliography / Webography	49
Appendix A — Service and Port Map	50
Appendix B — Main REST Endpoints	51
Appendix C — Glossary	53

List of Figures

- Figure 1.1 — Host organization logo
- Figure 1.2 — Host organization chart
- Figure 1.3 — Overview of the proposed solution
- Figure 3.1 — Global use case diagram
- Figure 3.2 — System sequence diagram: application submission (UC-2)
- Figure 3.3 — System sequence diagram: AI-assisted evaluation (UC-7, UC-8)
- Figure 3.4 — Activity diagram: application → selection → onboarding
- Figure 3.5 — Activity diagram: notifying a session's participants
- Figure 4.4 — Design sequence diagram: tracked session notification
- Figure 4.5 — Design sequence diagram: visibility-based session filtering
- Figure 4.6 — Component diagram of the platform
- Figure 5.1 — Sequence diagram: stateless JWT authentication
- Figure 5.2 — Login screen
- Figure 5.3 — Programme dashboard (back-office)
- Figure 5.4 — Visual Journey builder (back-office)
- Figure 5.5 — Session detail (Details / Activities tabs)
- Figure 5.6 — Evaluation dashboard and candidature review (with AI scoring)
- Figure 5.7 — Participants hub and invitation template (placeholders + tracking link)
- Figure 5.8 — Public programme page (front-office)
- Figure 5.9 — Application form and jury evaluation page (front-office)
- Figure 5.10 — End-of-studies project planning (Gantt chart)

List of Tables

Table 1.1	— Host organization fact sheet
Table 1.2	— Comparison of competing Tunisian offerings
Table 1.3	— Comparison of development methodologies
Table 3.1	— Identification of actors
Table 3.2	— List of major use cases
Table 4.1	— Description of domain classes
Table 4.2	— Data dictionary — Programme
Table 4.3	— Data dictionary — ProgrammePhase (Session)
Table 4.4	— Data dictionary — Candidature
Table 4.5	— Data dictionary — Evaluation
Table 4.6	— Data dictionary — Invitation
Table 5.1	— Software environment
Table 5.2	— Hardware environment
Table 5.3	— Service and port map
Table 5.4	— Project planning (sprints)
Table 5.5	— Test scenarios
Table A.1	— Extract of main REST endpoints

General Introduction

The digital economy has turned the **startup** into a central engine of value creation and employment, and **incubators** into the institutions tasked with supporting projects from idea to market. In Tunisia, the adoption of the **Startup Act** and the growing number of incubation and acceleration structures have pushed up, year after year, the number of programs that need to be managed. Each program involves a fairly complex choreography of actors and events: a call for applications, the intake and screening of files, the formation of juries, preselection and pitch sessions, the onboarding of the startups selected, several months of mentoring and training, and finally a demo day in front of investors.

There's something paradoxical about it, though: these same incubators, who help startups through their own digital transformation, mostly run their own programs in a fairly **artisanal** way — application forms scattered across third-party tools, selection grids kept in spreadsheets, juries contacted by email, agendas kept by hand, communications lost somewhere in an inbox. The result: the process slows down, mistakes pile up, consolidating results becomes a chore, and there's barely any traceability — right when funders and partners are asking for transparency and measurable outcomes.

It's in this context that this end-of-studies project was carried out, at **Medianet**: to **design and develop an integrated platform for managing incubation programs**, called **Medianet Incubateur**. The idea is to give program managers a single, coherent, and secure environment to design a program, receive and evaluate applications, run the sessions of its **journey** (or **parcours**), and manage — while keeping informed — everyone involved. Project carriers, juries, and the general public, for their part, each get an access experience suited to their role.

This project isn't just a matter of digitizing existing practices; it introduces a few ideas that genuinely shape the solution. First, a **unified, extensible session model**, capable of representing any stage of a program — application, preselection, pitch day, onboarding, incubation, demo day, training — without ever touching the database schema. Then, a **visibility** dimension that distinguishes public, internal, and private sessions. A **centralized validation layer**, too, which guarantees date consistency and the integrity of the journey. And finally, an **AI-assisted evaluation** that helps juries score applications more objectively, against the criteria specific to each program.

The central problem this project tries to solve can be put as a single question: **how do you design a single platform capable of modeling an incubation program as a structured, visual journey of typed, access-controlled sessions, while guaranteeing data reliability, role-based access security, and communication traceability?**

A second question, just as central, emerged as the project advanced: **how far can the evaluation of a pitch be made objective, reproducible and useful to the person being evaluated — without the video ever leaving the incubator's own infrastructure?** Asking a language model to "rate this pitch" is trivial and, as this report will show, unreliable: it produces confident prose with nothing underneath. The approach defended here is different. Everything that can be measured is measured from the signal itself — speech rate, loudness and its range, real silences from the waveform, hesitations aligned word by word — and the language model

is then asked to judge only against those measurements, following a rubric calibrated to the format of the pitch. What cannot be measured reliably, physical presence in particular, is explicitly demoted to weak evidence rather than dressed up as a fact. The whole pipeline is self-hosted: transcription, audio analysis and vision all run locally, so a founder's confidential pitch never leaves the premises.

Four chapters make up the rest of this report. **Chapter 1** lays out the general framework — host organization, existing solutions, the proposed answer, the development methodology behind it. **Chapter 3** turns that into requirements: actors, use cases, functional and non-functional needs, with the major use cases worked out in detail through dynamic diagrams. **Chapter 4** is where the object-oriented design takes shape — class diagram, data dictionary, sequence diagrams, component diagram. And **Chapter 5**, the last one, is about what actually got built: the work environment, the architecture, the interfaces, the test dataset, the planning, and the difficulties that came up along the way. A general conclusion closes things out and points toward what could come next.

Chapter 1 — General Framework

Introduction

Before getting into the platform itself, it's worth setting the scene. This chapter walks through the host organization, the problem the internship was meant to address, and an early look at what got built — followed by a look at what's already out there, both in Tunisia and beyond, and why it falls short. It ends with the development methodology chosen, and why it won out over the alternative.

1.1 Host Organization

1.1.1 General Presentation

Medianet is a Tunisian company specializing in digital services, web, and software solutions. Since it was founded in **1998**, the company has supported both public and private organizations in their digital transformation, notably through **web development, hosting, digital marketing, e-business, and software engineering.**



Figure 1.1 — Host organization logo

The table below summarizes the key facts about the host organization.

Item	Value
Company name	Medianet
Legal form	SARL
Year founded	1998
Head office	Ariana, Tunisia
Industry	Digital services / Web / Software
Main activities	Web, hosting, digital marketing, software engineering
Headcount	50 – 200 employees
Website	www.medianet.tn

Table 1.1 — Host organization fact sheet

1.1.2 Fields of Activity

Concretely, the host organization is active in several areas:

- **Web development and custom applications** for public and private clients ;
- **Hosting and managed services** for digital solutions ;
- **Digital marketing** and support for online presence ;

- **Innovation support**, in particular helping structure incubation and acceleration programs — precisely the setting this end-of-studies project fits into.

1.1.3 Company Organization

[Insert the company's organization chart]

Figure 1.2 — Host organization chart

Figure 1.2 shows the company's organization chart. It's structured into several departments, one of which hosted this end-of-studies project.

1.1.4 Host Department

The internship took place within the **[software engineering / innovation]** department, under the supervision of **[Company supervisor's name]**. This is the team that designs and builds the company's web applications, both for internal use and for its clients.

The table below summarizes the administrative framework of the internship.

Item	Value
Nature	End-of-studies project
Duration	6 months
Period	01-02-2026 – 31-07-2026
Academic supervisor	Slim Guermazi
Company supervisor	Yassine Mechich
Specialization	Software Architecture Engineering

1.2 Project Overview

This project consists of designing and developing **Medianet Incubateur**, a web platform whose purpose is to digitize the entire lifecycle of a startup incubation program. It brings together, in a single tool adapted to each role, the visual design of a program as a journey of typed, visibility-controlled sessions, online applications, jury evaluation assisted by artificial intelligence, the management of participants and organizations, and communications that can actually be tracked.

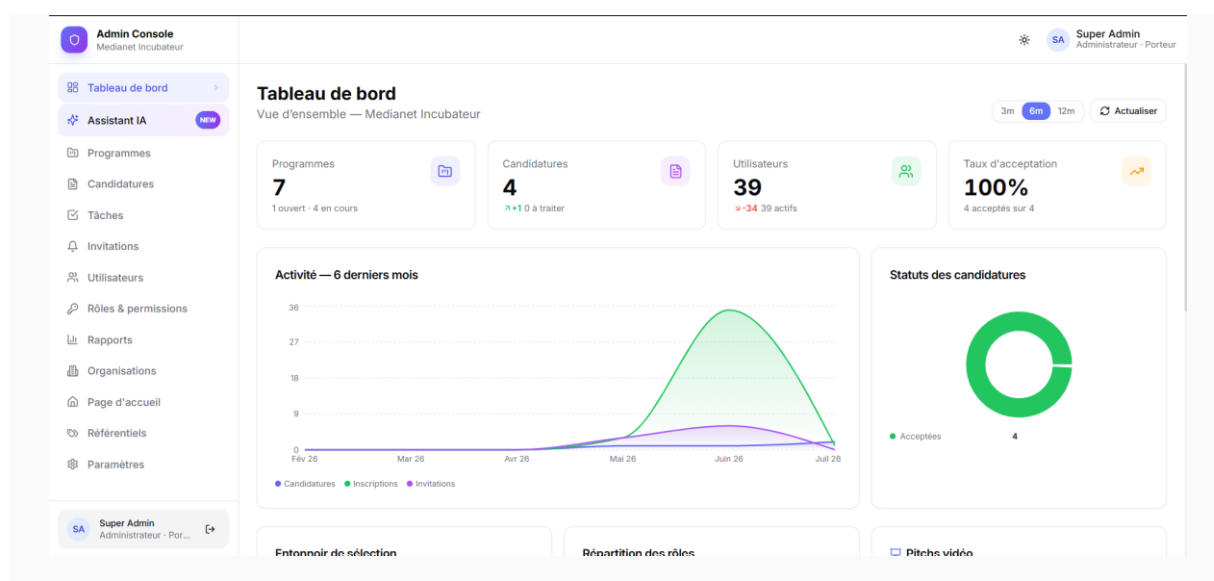


Figure 1.3 — Overview of the proposed solution

As shown in Figure 1.3, the solution rests on two distinct interfaces: a **back-office**, built for administrators and program managers, and a **front-office** aimed at project carriers, juries, organization members, and, more broadly, the general public.

1.3 Study of the Existing Landscape

1.3.1 Description of Existing Practices

Whether at the host organization or at comparable players in the Tunisian ecosystem, the management of incubation programs still largely relies on a patchwork of generic tools:

- application forms built with third-party tools, such as Typeform or Google Forms ;
- selection and scoring grids that live in spreadsheets (Excel, Google Sheets) ;
- agendas kept by hand in shared calendars ;
- communications handled by email and instant messaging, with no real centralized archive ;
- progress reports that have to be recalculated manually for every review.

That's the case for several Tunisian incubators — **Flat6Labs Tunis**, **Wiki Start Up**, **B@Labs (BIAT)**, **Impact Partner**, **The Dot**, **Redstart Tunisie** — which publish their calls for applications and run their cohorts in a fairly similar way: a presentation page, an external form, and follow-up that stays manual internally. The national **Startup Tunisia / Startup Act** portal follows a different logic, since it focuses on the legal labeling of startups rather than on the operational management of a program's journey.

The table below compares, feature by feature, what these solutions typically cover against what this project aims for.

Feature	Generic incubator portals	National labeling portal	Medianet Incubateur
Public program presentation	Yes	Partial	Yes

Feature	Generic incubator portals	National labeling portal	Medianet Incubateur
Online application form	Yes (often external)	No	Yes (per program)
Visual program journey	No	No	Yes
Typed sessions with activities	No	No	Yes
Session visibility control	No	No	Yes
Jury evaluation (weighted criteria)	Rare / external	No	Yes
AI-assisted scoring	No	No	Yes
Participant directory & tracked invitations	Partial	No	Yes
Multi-role access (admin/jury/carrier)	Partial	Partial	Yes

Table 1.2 — Comparison of competing Tunisian offerings

1.3.2 Critique of the Existing Landscape

This analysis of the existing landscape brings out a few weaknesses that keep coming up:

- **Fragmented tooling.** Applications, the selection grid, jury scoring, the agenda, and communications each live in their own corner — separate files, different inboxes. Nothing is centralized, and the same data keeps getting copied back and forth between tools.
- **No single source of truth for the journey.** The program's calendar — its sessions, their order, their nesting (a training day slipped inside an incubation period), their dependencies — isn't modeled anywhere. In practice, it only exists in the program manager's head and in a handful of scattered documents.
- **Manual, non-reproducible evaluation.** Juries are contacted by email, receive applications as attachments, then score them on paper or in improvised spreadsheets. Consolidating weighted scores by criterion, by jury, and by session quickly becomes tedious — and re-evaluating the same candidate in a later session, a pitch day following a preselection for instance, is even harder.
- **Weak, informal access control.** Some sessions are purely internal — planning, jury deliberation, score recalibration — and should never be visible to startups or the public. Yet nothing structurally prevents it. Conversely, an external jury invited to a single private session shouldn't, in theory, get access to the rest of the program.
- **Insufficient traceability of communications.** When a manager sends an invitation or an update, there's no reliable record of who was contacted, when, with what message, or even whether the email actually reached its destination.

- **No consolidated metrics.** Funnel metrics (applications received, evaluated, accepted), jury progress, sector breakdown, or the ranking of top candidates — all of this has to be recalculated by hand for every report.

Add it all up and these weaknesses carry a real cost: **time lost** on repetitive manual tasks, **errors** in transcription and score consolidation, **delays** in decisions, a certain **frustration** among juries and project carriers facing a process they don't really control, and a lot of **difficulty** producing the transparent reporting that funders and partners expect.

1.4 Proposed Solution

Starting from these observations, the proposed solution takes the form of a **single, secure web platform**, capable of:

- modeling a program as a structured, visual journey of typed sessions ;
- receiving and managing applications online ;
- supporting reproducible, per-session jury evaluation, assisted where relevant by artificial intelligence ;
- managing every participant and organization, and communicating with them in a traceable way ;
- strictly enforcing business rules — date validation, session visibility — both server-side and in the interface ;
- clearly distinguishing roles: administrator, program manager, project carrier, jury, mentor, and general public.

Among these pillars, one carries most of the project's originality and deserves to be stated plainly. **The platform does not merely digitize the presentation day; it makes the evaluation of a pitch measurable.** Where an incubator today relies on a jury's subjective impression, jotted down after the fact, the platform produces evidence: how fast the founder actually spoke, how loud and how varied the voice was, where the real silences fell, how many hesitations slipped out and exactly when. Those measurements — not an impression — are what the scoring is built on, and every remark returned to the founder is anchored to a timestamp they can replay. This turns a one-off verdict into something a project carrier can train against, which is why the model distinguishes training pitches, where progress is tracked from one attempt to the next, from the final pitch that counts.

- turning the presentation day into a coaching tool: transcribing each pitch automatically, measuring the delivery objectively from the audio signal, and returning a scored, time-coded, actionable analysis ;
- running that entire analysis on self-hosted models, so no candidate video ever leaves the infrastructure ;

The following chapters go through each of these pillars in turn: the visual journey builder, typed and visibility-controlled session management, online applications, AI-assisted evaluation (nicknamed « **Medi** »), participant management with tracked invitations, and administrable reference catalogs.

1.5 Development Methodology

Choosing the right development methodology matters just as much as everything else, especially for a project whose requirements keep evolving along the way. Before justifying the choice made, this section briefly presents two reference approaches.

1.5.1 The Waterfall Cycle

The **waterfall** model moves sequentially: each phase — specification, design, implementation, testing, deployment — has to be finished before moving on to the next. When requirements are locked in from the start, that predictability is a real asset. But as soon as requirements get refined along the way, the model shows its limits, since going back becomes costly.

1.5.2 Agile Methods (Scrum)

Agile methods — **Scrum** being the best known — work quite differently: development is broken down into **short iterations**, sprints, each delivering a functional increment that gets reviewed with stakeholders before moving on to the next. Adapting to requirements becomes continuous, problems surface earlier, and collaboration between developers and business stakeholders tightens up.

1.5.3 Methodology Chosen

For this project, **the requirements weren't fully known upfront** — they became clearer as the work progressed. The session model, the visibility rules, the evaluation flow: each of these matured over several iterations, in discussion with the host organization. A waterfall cycle, which assumes everything is locked down from day one, simply wouldn't have fit that reality. That's why an **agile, iterative, incremental approach** was chosen instead: the work moves forward in short increments, each delivering a coherent, deployable vertical slice — back-end and front-end together — continuously integrated and validated before moving on.

UML, for its part, is used throughout the project as a **design and communication aid**: it helps clarify actors, use cases, the domain model, and the main dynamic flows, without ever trying to lock down an exhaustive specification upfront. This lightweight, model-driven agility is close in spirit to the **Two-Track Unified Process (2TUP)** and to **Scrum** practices.

The table below summarizes this comparison.

Criterion	Waterfall	Scrum (chosen)
Adaptation to evolving requirements	Low	High
Value delivery	At the end of the project	At every iteration
Risk of costly rework	High	Low
Suited to still-uncertain requirements	No	Yes

Table 1.3 — Comparison of development methodologies

Concretely, the approach adopted rests on the following points:

- **Product backlog.** Requirements are expressed as **user stories** per actor, then prioritized.

- **Increments / sprints.** Each increment targets a coherent set of user stories — first the program builder, then applications, evaluation, AI scoring, visibility, and finally participants.
- **Continuous integration and delivery.** Every change goes through static typing (``tsc``), a build (``mvn`` or Next.js build), and deployment as containers (``docker compose build`` / ``up``).
- **Microservice isolation.** Each business context is developed and deployed independently, with its own database.
- **Verification.** Manual test scenarios and end-to-end checks accompany every increment, systematically covering each session type and each role.

Conclusion

So that's the backdrop: a host organization, a problem that turned out to be real rather than invented, an early sketch of the platform, and the methodology behind how it got built. From here, the next step is turning that sketch into requirements. That's where Chapter 3 picks up.

Chapter 2 — Theoretical Foundation

Introduction

The platform described in this report does not merely store and display data: it measures a human performance and issues a judgement on it. That ambition cannot rest on intuition. This chapter therefore sets out the scientific groundwork the solution is built on — how speech is transcribed automatically, how the qualities of a voice can be quantified from the signal itself, what a vision model can and cannot be trusted to see, and under what conditions a language model may reasonably be used as an evaluator. Each notion introduced here is used later: Chapter 4 designs around it, Chapter 5 implements it, and Chapter 6 measures whether it held.

2.1 Automatic Speech Recognition

2.1.1 From acoustic models to end-to-end architectures

Automatic Speech Recognition (ASR) converts an audio signal into text. Historically the task was split into separate components — an acoustic model mapping sound to phonemes, a pronunciation lexicon, and a language model reordering the candidates into plausible sentences. Each component was trained apart, and errors accumulated along the chain. Modern systems replace this pipeline with a single end-to-end neural network trained to map audio directly to characters, which removes the hand-built lexicon and lets the model learn punctuation and casing on its own.

Whisper, published by OpenAI in 2022, belongs to this family. It is an encoder–decoder Transformer: the encoder consumes a log-Mel spectrogram — a time–frequency representation of the sound — and the decoder generates the text token by token, each token conditioned on those already produced. Its distinguishing feature is not its architecture but its training regime: it was trained on roughly 680 000 hours of weakly supervised multilingual audio collected from the web, which is what makes it robust to accents, background noise and recording conditions without any fine-tuning. **This robustness is the reason it was selected here: a pitch is filmed in a room, not in a studio.**

2.1.2 The normalisation bias and its consequence for this project

That same training regime carries a bias that turns out to be decisive for this work. **Whisper was trained to produce clean, readable prose — the kind of text found in subtitles and transcripts — and it therefore learns to omit what a human transcriber would omit: the hesitations.** Filled pauses ("uh", "um"), false starts and repetitions are systematically absent from its output. For a subtitling task this is a feature. For an application whose stated goal is to detect hesitation, it destroys the very signal being sought.

Two properties of the decoder make it possible to work around this. First, the decoder is autoregressive and conditioned on its own context, so a prefix supplied to it (the initial prompt) influences the register of everything that follows. Second, that prompt is not transcribed; it only conditions the style. Priming the decoder with a deliberately disfluent sample therefore shifts it toward verbatim transcription. Chapter 5 details this technique and Chapter 6 measures its effect.

A second property is exploited later: word-level timestamps. By aligning the decoder's cross-attention with the input frames, Whisper can return a start and end time for every word, together with the probability it assigned to it. These three quantities — the word, its span and its confidence — are the raw material for the disfluency measurements defined in section 2.2.3.

2.2 Objective measurement of the speech signal

Transcription tells what was said. It says nothing about how. Everything that concerns delivery — whether the voice carries, whether it is monotonous, whether the speaker hesitates — has to be measured on the waveform, independently of any model. This section defines the quantities used.

2.2.1 Loudness: the EBU R 128 standard

Perceived loudness is not the physical amplitude of a signal: the ear is not equally sensitive at all frequencies. The EBU R 128 recommendation, together with ITU-R BS.1770, defines a measurement that approximates perception. The signal is first filtered by a "K-weighting" curve, which attenuates low frequencies and slightly emphasises high ones to mimic the ear's response; the mean square of the filtered signal is then integrated over time and expressed in LUFS (Loudness Units relative to Full Scale), a logarithmic scale where 0 LUFS is the maximum and values are therefore negative.

Two figures derived from this standard are used in this project. The **integrated loudness** summarises the average level of the whole recording: broadcast practice targets about -23 LUFS, and a pitch measured well below that — around -30 LUFS — is objectively too quiet to be heard comfortably. The **loudness range (LRA)**, expressed in LU, measures the spread between the quiet and loud passages. A low LRA is the signature of a monotonous delivery; a speaker who varies intensity to emphasise a point produces a higher one. These two numbers replace the subjective judgement "he spoke too softly and in a flat voice" with a reproducible measurement.

2.2.2 Silence detection and its interaction with ASR

A pause is a silence longer than a given threshold. Detecting one requires two parameters: a level below which the signal is considered silent (in dB) and a minimum duration below which the gap is a mere articulatory break rather than a pause. From the detected intervals follow the number of pauses, the longest one, and the speaking ratio — the share of the recording that is actual speech.

One methodological point deserves emphasis, because it caused a real error in this project. Most ASR systems, Whisper included, apply a **Voice Activity Detection (VAD)** filter that removes silence before transcription. As a consequence, the gaps between the transcribed segments are not the silences of the original recording — they have already been eliminated. Deriving pauses from transcript segments therefore yields nothing. Silences must be measured on the raw audio, independently of the ASR. This is a general principle: **a measurement must be taken on the source, never on an artefact that has already been transformed.**

2.2.3 Word-level alignment and disfluency markers

Psycholinguistics distinguishes several markers of disfluency: filled pauses ("uh", "um"), prolongations (a sound held abnormally long), repetitions and repairs (false starts), and hedging expressions ("I think", "maybe") which weaken assertion. Uptalk — a statement delivered with the intonation of a question — belongs to the same family. These markers are studied as indices of cognitive load and of the speaker's confidence, which is precisely what a jury perceives without being able to quantify it.

Word-level timestamps make several of them measurable. A prolongation, for instance, can be defined as a short word whose duration exceeds a threshold. This definition is only valid, however, if one corrects for a property of the aligner: **the end of a word is placed where the next one begins, so any silence that follows a word is charged to that word.** A word followed by a pause therefore looks stretched. Subtracting the silences measured in section 2.2.2 from each word's span restores the voiced duration, which is the quantity actually of interest. Chapter 6 quantifies the difference between the naive measurement and the corrected one.

2.3 Vision–language models and the limits of visual evidence

Physical presence — posture, gaze, gesture — is part of what a jury judges. Vision–language models make it partially accessible: an image encoder projects a picture into the same representation space as a language model, which can then describe the scene in natural language. A compact model of this family, executed locally, is used in this project to describe sampled frames of the pitch.

Three limits govern how such a description may legitimately be used. First, **a sampled frame is an instant, not a behaviour**: one image every forty seconds shows postures, not habits, and a trait observed once may be an accident of timing. Second, decoding is **stochastic**: at a non-zero temperature the model samples from a probability distribution, so the same image can produce different descriptions from one run to the next. Reproducibility therefore requires fixing the temperature and the random seed. Third, these models are prone to **hallucination** — producing a plausible description unsupported by the pixels — a risk that grows when the subject occupies a small part of the frame, which is exactly the case for a speaker filmed on a stage. These three limits justify the design rule adopted in Chapter 4: visual observations are treated as weak evidence, and a trait seen in a single frame may never, on its own, justify advice.

2.4 Large language models as evaluators

2.4.1 Principle and known biases

A large language model (LLM) is a Transformer trained to predict the next token over very large corpora. Its instruction-tuned variants can be asked to apply a grading rubric to a text, an approach known in the literature as "LLM-as-a-judge". Its appeal is obvious — it produces an argued, structured judgement in seconds — and so are its weaknesses, which are documented and were all observed during this project.

The first is **hallucination**: the model states, fluently and confidently, facts that are not in its input. The second is **sycophancy**: a tendency to be agreeable, which on an evaluation task translates into systematic leniency. The third is **sensitivity to the prompt's formulation**: a rubric expressed as rigid caps does not produce a stricter grader but a degenerate one, as Chapter

6 shows experimentally. The fourth is **non-determinism**, identical in nature to that of the vision model. Taken together, these biases forbid asking an LLM to "rate this pitch" and taking the answer at face value.

2.4.2 Grounding and calibration

The approach defended in this report follows from those limits and rests on two principles. **Grounding** consists in never asking the model for a fact it could invent: everything factual — speech rate, loudness, pauses, hesitations and their timestamps — is measured beforehand by the deterministic methods of section 2.2, and the model is asked only to interpret those measurements against the programme's criteria. Its role is reduced from witness to analyst. **Calibration** consists in verifying that the resulting scale actually discriminates: a rubric that awards the same grade to an excellent pitch and to a poor one carries no information, whatever the quality of its prose. Calibration is therefore an experimental question, settled on a reference corpus, and Chapter 6 treats it as such.

One structural precaution completes them: the overall score is not requested from the model but derived from its per-dimension scores. This prevents a holistic "impression" from overriding the detail, and makes the grade reconcilable with the profile shown to the user.

2.5 Architectural and security foundations

The functional requirements also rest on a few established notions, recalled here briefly. A **microservices architecture** decomposes an application into independently deployable services, each owning its data; it fits this project because the media pipeline has resource needs — CPU, models, ffmpeg — that have nothing in common with those of a CRUD service, and must be able to fail or be redeployed without taking the platform down. A **service registry** and an **API gateway** provide, respectively, discovery and a single entry point.

Security rests on the **JSON Web Token (JWT, RFC 7519)**, a signed and self-contained token that carries the user's identity and rights, allowing each service to authorise a request without a shared session. It is combined with **Role-Based Access Control (RBAC)**, extended here with per-user permissions and role inheritance. Finally, **Server-Sent Events (SSE)** provide a unidirectional server-to-client channel over HTTP, used for two purposes: propagating a rights change to a connected user, and streaming the progress of the analysis pipeline, whose duration makes a synchronous request unacceptable.

Conclusion

This chapter established what the rest of the report relies on: an ASR system robust but biased toward clean prose, whose bias must be worked around to observe hesitation; a set of standardised acoustic measurements — loudness, loudness range, silences — that objectify delivery; word-level alignment, usable only once corrected for the silence charged to each word; a vision model whose output is weak, stochastic evidence; and a language model that is a credible evaluator only if it is grounded in prior measurements and calibrated on a reference corpus. These principles are not decoration: each one corresponds to an error actually made and corrected during the project, as Chapter 6 documents. Chapter 3 now turns them into requirements.

Chapter 3 — Requirements Analysis and Specification

Introduction

Turning a vision into something buildable means writing it down precisely. That's the job of this chapter: functional and non-functional requirements, the actors involved, the use cases they trigger, and — for the ten that matter most — a full breakdown through textual descriptions, system sequence diagrams, and activity diagrams.

3.1 Project Vision

The vision behind **Medianet Incubateur** fits in a few lines:

*Give incubators a single platform that turns a program into a **structured, visual journey** of typed, access-controlled sessions; lets candidates **apply online**; lets juries **evaluate** (assisted by artificial intelligence); and keeps every participant **informed** through traceable communications — all while guaranteeing **strict data validation** and a **clear separation of roles**.*

3.1.1 Functional Requirements

For clarity, functional requirements have been grouped by module.

M1 — Authentication and Accounts

- Sign-up and sign-in with **JWT** ; password reset.
- User profile management, differentiated by role (carrier, jury, mentor).
- Administration of users, their **roles**, and their **fine-grained permissions**.

M2 — Programmes and Journeys

- Create, view, update, and delete programmes, along with their metadata (title, description, type, dates, sectors, eligible organization types, key figures, partners) and a **customizable application form**.
- Manage **evaluation criteria** (name, weight, maximum score) per programme.
- Build the visual **journey**: create **sessions** (day or range), **nest** day-sessions inside ranges, organize them into **lanes**, define their **type, visibility, dates, location, owners** and **guests**, and set **per-session criteria weights**.
- Edit an **activity agenda** (hourly grid) for day-sessions ; reuse **presets** and full **journey templates**.
- **Automatically synchronize the programme status** based on the dates and statuses of its sessions.

M3 — Applications

- Submit an application to an open programme through the programme's own form, linking the carrier's **organization** and **team**.
- Manage an application's **status lifecycle** (pending, under evaluation/review, accepted, rejected).
- View the **full detail** of an application, including the answers submitted.

M4 — Evaluation

- **Assign juries** to an evaluation session.
- **Score** an application against weighted criteria ; allow **re-evaluation per session** (a candidate is scored afresh in every evaluation session).
- Run an **AI score (Medi)** producing per-criterion notes, a weighted score, a recommendation, and a summary ; let administrators score as well.
- Provide a **consolidated dashboard** with ranking and export.

M5 — Participants and Communications

- List the **people associated** with a programme (organizations, members, carriers, juries).
- **Add already-registered users** or **invite people by email**.
- Compose a **personalized invitation** from a template with placeholders and a **tracking link** ; **preview before sending**.
- Send **per-session notifications** to the relevant participants and keep a **full archive** of every send.

M6 — Visibility and Access Control

- Set a session's **visibility**: **Visible**, **Hidden (internal)**, or **Private**.
- **Enforce** visibility server-side based on the requester's **role** and **invitations**, and reflect it in the interface.

M7 — Reference Catalogs

- Manage **catalogs** (organization types, programme sectors, session types) used by the forms.

M8 — Presentation Day and AI Pitch Analysis

- Flag any session for AI video analysis with a simple checkbox, independently of the session type.
- Let a carrier upload several training videos and one final pitch video (up to 2 GB each), stored in MinIO.
- Transcribe the pitch automatically and time-code it, keeping real hesitations rather than a cleaned-up text.
- Measure delivery objectively from the waveform: speech rate, loudness, loudness range, real pauses and speaking ratio.
- Detect unconscious speech habits word by word: filler sounds, drawn-out sounds, repetitions, hedging and tag questions.
- Describe physical presence from sampled frames using a locally executed vision model, treated as weak evidence.
- Score the pitch per dimension against the programme criteria, after detecting its format (demo day, competition, investor, internal).
- Return time-coded key moments, a criterion mapping and a costed coaching plan (impact in points, effort, ready-to-say wording).
- Stream the pipeline stages live to the interface with time estimates learned from real runs.
- Show progress analytics at three levels: overall, per programme and per training phase.

3.1.2 Non-Functional Requirements

Quality	Requirement
Security	Stateless JWT authentication at the gateway and in each service ; role and permission authorization (‘module:action’) ; role-based data filtering, notably for session visibility.
Scalability	Independent, stateless microservices behind a gateway with service discovery ; one database per service .
Maintainability & extensibility	Well-scoped business contexts ; DTO layers ; a centralized validation layer with explicit error codes ; an extensible session model (new types without schema rework).
Usability	Two dedicated interfaces ; a readable, tabbed session interface (details first, activities kept separate) ; confirmation before adding and preview before editing for sensitive actions.
Performance	Lazy loading, parallelized requests, no N+1 cross-service calls thanks to denormalized display fields.
Reliability	Resilient cross-service calls (graceful degradation), structured error handling, reproducible deployments.
Portability	Fully containerized solution (Docker), configuration driven by environment variables.
Localization	French-language interface, adapted to the local context.

3.2 Requirements Specification

3.2.1 Identification of Actors

An **actor** is a role played by a user, or by an external system, that interacts with the platform.

Actor	Type	Description
Administrator	Human (primary)	Full control over programmes, journeys, criteria, applications, evaluations, users, roles, catalogs, and communications.
Program Manager	Human	A restricted administrator, scoped by permission slugs (‘programmes:*’,

Actor	Type	Description
		`candidatures.*`, ...) to a programme's operations.
Project Carrier (Porteur)	Human (primary)	Founder who applies to programmes, manages their organization, and follows the journey and their tasks.
Jury	Human (primary)	Evaluates the applications assigned to them, per evaluation session, optionally assisted by AI.
Mentor	Human	Supports the selected startups ; views the sessions and tasks assigned to them.
Organization Member	Human	Member of a participating organization ; follows programme updates.
Visitor	Human	Anonymous user who browses public programmes and the public journey (Visible sessions only).
Medi (LLM)	External system	The AI service that scores applications.
SMTP Relay (Brevo)	External system	Delivers invitation and notification emails.

Table 3.1 — Identification of actors

3.2.2 Identification of Use Cases

The diagram below summarizes how these actors interact with the platform.

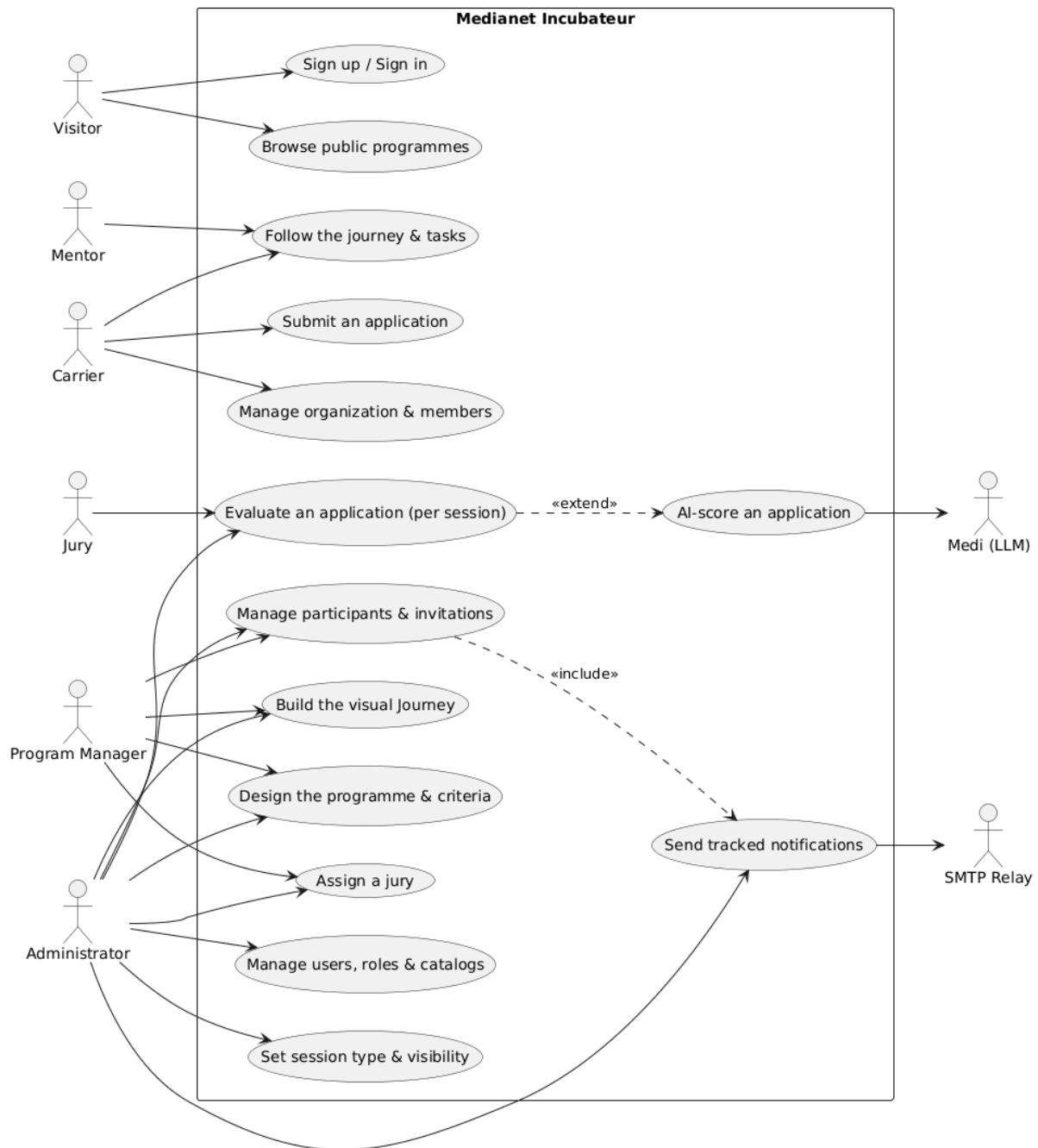


Figure 3.1 — Global use case diagram

Figure 3.1 shows the full set of use cases. Ten of them were selected for detailed specification ; they are listed in Table 3.2.

Code	Use Case	Primary Actor
UC-1	Authenticate	All registered actors
UC-2	Submit an application	Carrier
UC-3	Design a programme and its criteria	Administrator
UC-4	Build the visual Journey	Administrator

Code	Use Case	Primary Actor
UC-5	Set a session's type and visibility	Administrator
UC-6	Assign a jury	Administrator
UC-7	Evaluate an application (per session)	Jury
UC-8	AI-score an application	Jury / Administrator
UC-9	Manage participants and send invitations	Administrator
UC-10	Follow a programme	Carrier / Jury

Table 3.2 — List of major use cases

3.3 Detailed Requirements Specification

3.3.1 Textual Description of Major Use Cases

UC-1 — Authenticate

Field	Description
Actors	All registered actors
Goal	Obtain an authenticated session (JWT) to access role-protected features.
Preconditions	The user has an account.
Postconditions	A signed JWT is issued and stored ; the user is redirected to their dashboard.
Main flow	1. The user opens the login page. 2. They enter their email and password. 3. The system validates the credentials. 4. The authentication service issues a JWT containing the user's roles and permissions. 5. The front-end stores the token and loads the appropriate workspace.
Alternatives	3a. Invalid credentials → an error message is shown.
Exceptions	The authentication service is unavailable → a technical error is displayed.

UC-2 — Submit an Application

Field	Description
Actors	Carrier
Goal	Apply to an open programme.

Field	Description
Preconditions	The programme is open (its application session is active and the deadline hasn't passed) ; the carrier is authenticated.
Postconditions	A `Candidature` is created with status *PENDING* and linked to the carrier's organization.
Main flow	1. The carrier opens the programme page. 2. They start an application and fill in the programme's own form. 3. They link or create their organization and team. 4. They submit. 5. The system checks eligibility and the deadline, creates the application, and confirms.
Alternatives	5a. The deadline has passed → submission is refused. 5b. Required fields are missing → validation errors are returned.
Exceptions	The candidature service is unavailable → the submission isn't recorded and the user is notified.

UC-3 — Design a Programme and Its Criteria

Field	Description
Actors	Administrator, Program Manager
Goal	Create and configure a programme and its evaluation criteria.
Preconditions	The actor is authenticated and authorized (programmes:update).
Postconditions	The programme and its criteria are persisted.
Main flow	1. The actor creates a programme and fills in its information (dates, sectors, eligible organization types, key figures). 2. They define the application form. 3. They add weighted evaluation criteria. 4. They save.
Alternatives	1a. Dates are inconsistent → a validation error is returned.
Exceptions	—

UC-4 — Build the Visual Journey

Field	Description
Actors	Administrator, Program Manager
Goal	Model the programme as a sequence of sessions and activities.

Field	Description
Preconditions	The programme exists ; the actor is authorized.
Postconditions	The sessions, their nesting, and their activities are persisted ; the programme status may be recomputed.
Main flow	1. The actor adds a session (day or range). 2. They set its type, visibility, dates, location, and participants. 3. They optionally nest a day-session inside a range. 4. They add an activity agenda to a day-session. 5. The validation layer checks the dates, overlaps, and activity constraints. 6. The actor confirms the creation and previews sensitive edits.
Alternatives	5a. A date violates a rule → an explicit error code is returned (e.g. SESSION_OVERLAP_DETECTED). 5b. Moving a range would push a nested day outside of it → the nested days are carried along once confirmed.
Exceptions	—

UC-5 — Set a Session's Type and Visibility

Field	Description
Actors	Administrator
Goal	Define a session's type and who can see it.
Preconditions	The session exists.
Postconditions	The type and visibility are persisted ; visibility is enforced server-side.
Main flow	1. The actor opens the session. 2. They pick a type (application, preselection, pitch day, ...). 3. They pick a visibility (Visible / Internal / Private). 4. They save ; the change is previewed beforehand.
Alternatives	—
Exceptions	—

UC-6 — Assign a Jury

Field	Description
Actors	Administrator, Program Manager
Goal	Designate the juries of an evaluation session.

Field	Description
Preconditions	An evaluation session exists ; the people to assign are known.
Postconditions	Jury assignments are created (with a tokenized evaluation link).
Main flow	1. The actor opens the evaluation session. 2. They select juries from the directory or by email. 3. They confirm the assignment. 4. The system records the assignments and may notify the juries.
Alternatives	—
Exceptions	—

UC-7 — Evaluate an Application (Per Session)

Field	Description
Actors	Jury, Administrator
Goal	Score an application for a given evaluation session.
Preconditions	The jury is assigned to the session.
Postconditions	An 'Evaluation' keyed by (candidature, jury, session) is stored ; the consolidated score is updated.
Main flow	1. The jury opens an assigned application. 2. They score each weighted criterion and add a comment. 3. They may run Medi and reuse its notes. 4. They submit.
Alternatives	3a. The jury disagrees with the AI and adjusts the notes manually.
Exceptions	—

UC-8 — AI-Score an Application

Field	Description
Actors	Jury, Administrator (system actor: Medi)
Goal	Obtain an AI-proposed score for an application.
Preconditions	The application exists ; the caller is authorized.
Postconditions	The AI score is displayed (not persisted until reused/submitted).
Main flow	1. The actor requests an AI score. 2. The AI service fetches the application, the criteria, and the organization/team context. 3. It builds

Field	Description
	a prompt and calls the LLM. 4. It parses the JSON result (per-criterion notes, weighted score, recommendation, summary). 5. It returns the result to the interface.
Alternatives	4a. The LLM's answer can't be parsed → a safe fallback is returned.
Exceptions	The LLM is unavailable → an error is shown and manual scoring remains possible.

UC-9 — Manage Participants and Send Invitations

Field	Description
Actors	Administrator
Goal	View the people associated with a programme and invite them via a tracked email.
Preconditions	The actor is authorized.
Postconditions	Invitations are sent and archived (status SENT/FAILED).
Main flow	1. The actor opens the Participants view. 2. They browse the directory (organizations, members, carriers, juries). 3. They add registered users or type in emails. 4. They personalize the invitation template (placeholders, tracking link) and preview it. 5. They send ; every send is archived.
Alternatives	5a. An email fails → it's archived with status FAILED.
Exceptions	—

UC-10 — Follow a Programme

Field	Description
Actors	Carrier, Jury
Goal	Follow a programme's journey, updates, and personal tasks.
Preconditions	The actor is authenticated and linked to the programme.
Postconditions	—
Main flow	1. The actor opens the programme. 2. They see the journey (Visible sessions and, for invited users, the sessions they're invited to). 3. They check their tasks and updates.
Alternatives	—

Field	Description
Exceptions	—

3.3.2 System Sequence Diagrams of Major Use Cases

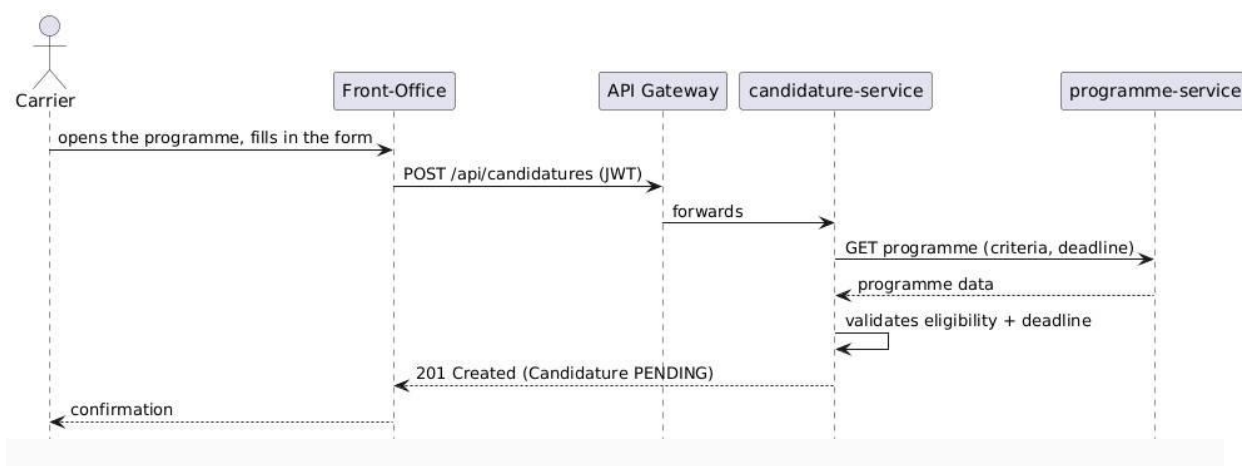


Figure 3.2 — System sequence diagram: application submission (UC-2)

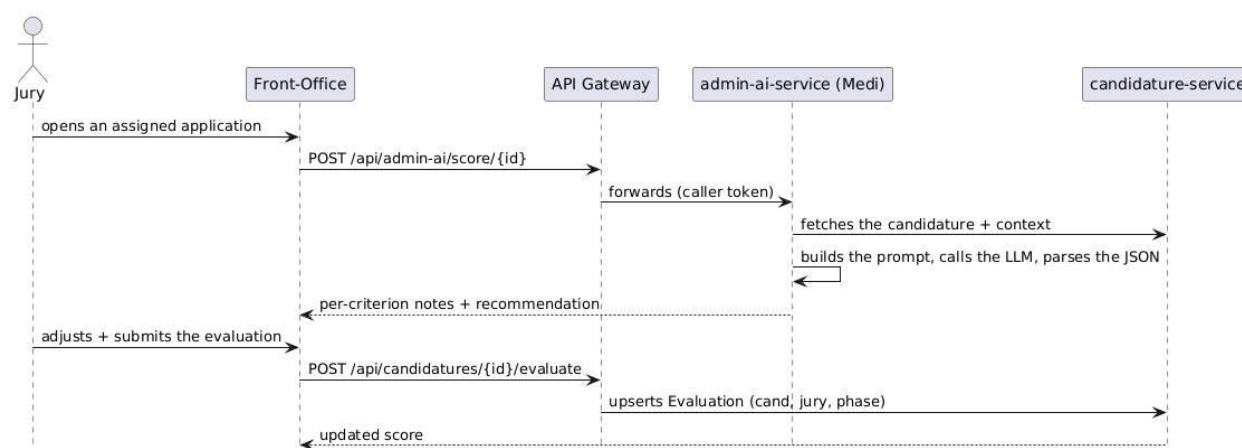


Figure 3.3 — System sequence diagram: AI-assisted evaluation (UC-7, UC-8)

3.3.3 Activity Diagrams

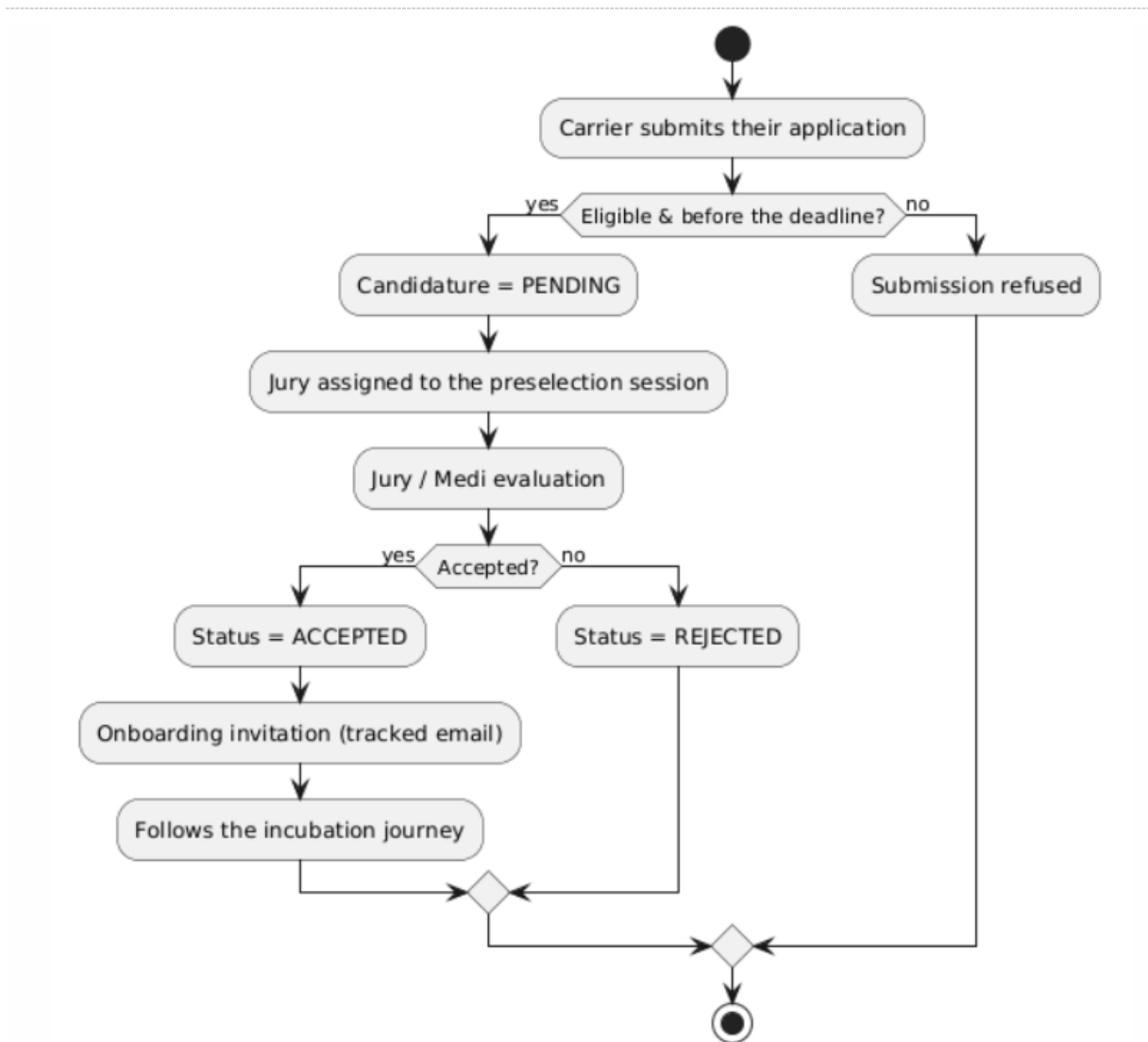


Figure 3.4 — Activity diagram: application → selection → onboarding

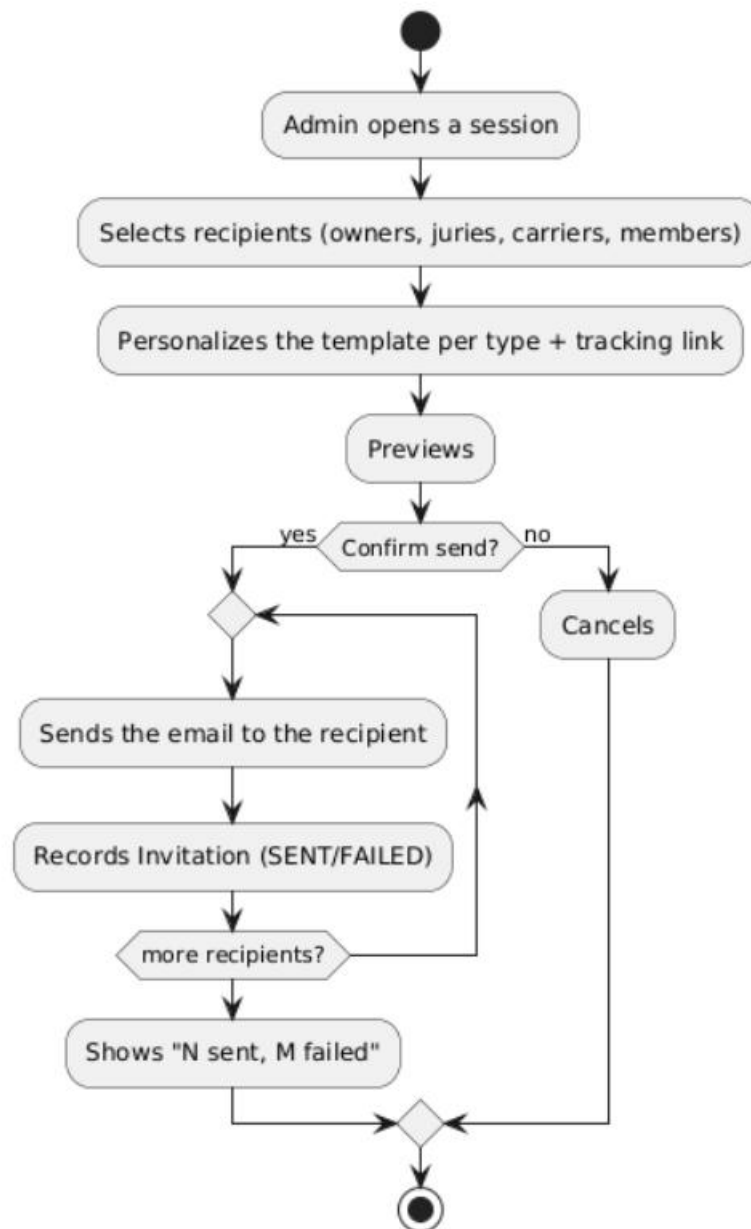


Figure 3.5 — Activity diagram: notifying a session's participants

Conclusion

Seven functional modules, a handful of measurable non-functional requirements, nine actors, ten use cases worked out in detail — that's what came out of turning the vision into something concrete. It's also, from here on, the specification the design has to respect. Chapter 4 takes it from there.

Chapter 4 — Design

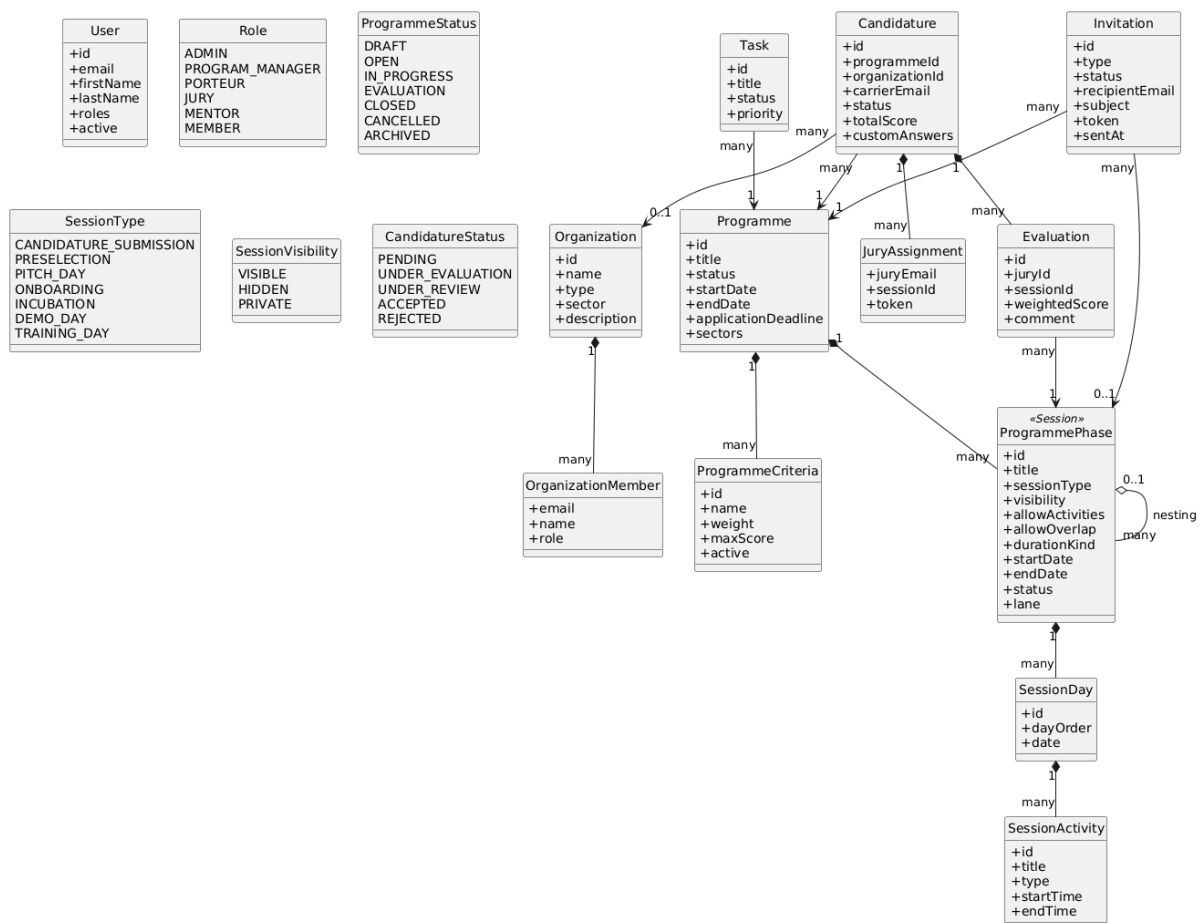
Introduction

Requirements are one thing; deciding how the system is actually structured is another. This chapter makes that jump, through the **design class diagram** and its data dictionary, a set of **design sequence diagrams** showing how the major use cases play out in practice, and the **component diagram** that maps the platform's deployable pieces.

4.1 Class Diagram

4.1.1 Design Class Diagram

The domain model revolves mainly around the **Programme** and its **sessions** — the unified 'ProgrammePhase' entity — as well as the **Candidature** and its **Evaluations**. Added to this are the **User/Organization** identities, and two cross-cutting entities, **Invitation** and **Task**.



The domain model is presented in three complementary figures. **Figure 4.1** gives the core of the platform, grouped into three coherent blocks: accounts and organizations, the programme and its journey of typed sessions, and applications with their evaluation. **Figure 4.2** isolates the pitch submission, which carries the video, its cached transcript and the result of the analysis. **Figure 4.3** finally details the structure of that analysis result, whose richness is what distinguishes this platform from a simple grade.

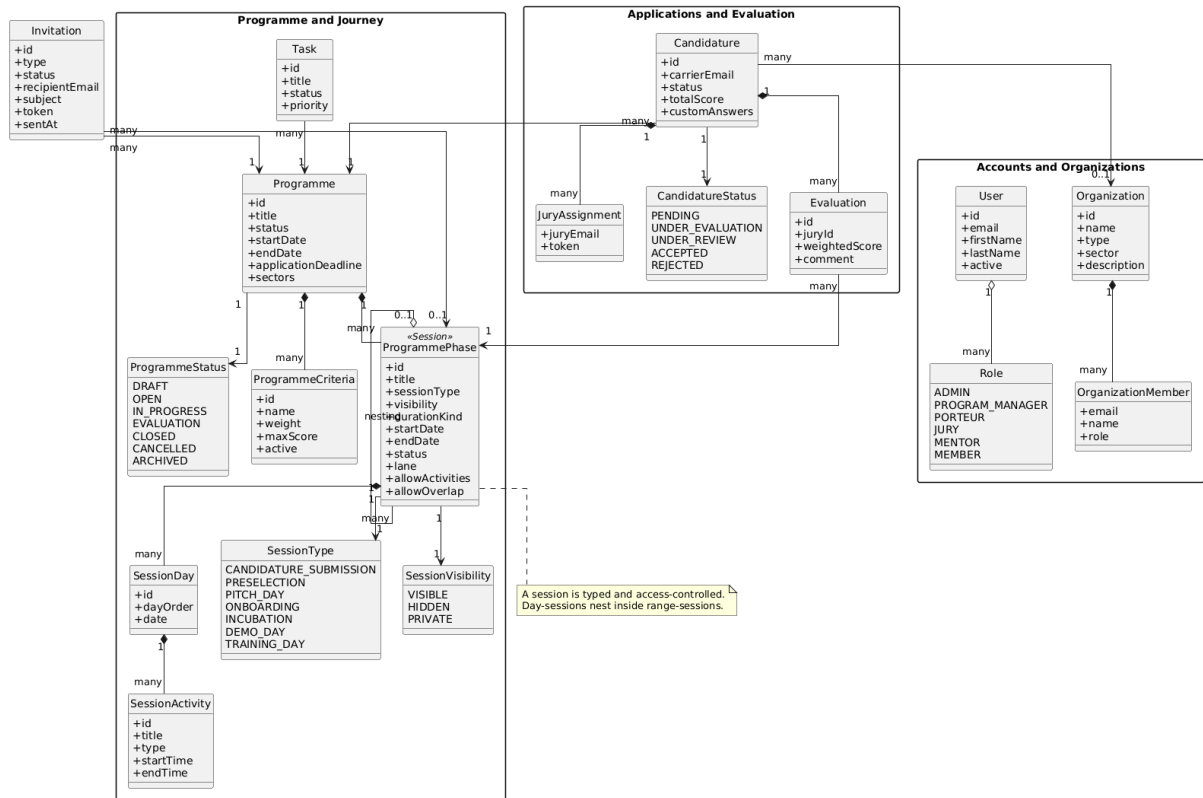


Figure 4.1 — Design class diagram: core domain

The session entity is named **ProgrammePhase** for historical reasons — the class kept the name of an earlier, simpler concept — but it represents a session in every sense described in Chapter 3: typed, access-controlled, datable and nestable. Its self-association carries that nesting: a day-session belongs to a range-session.

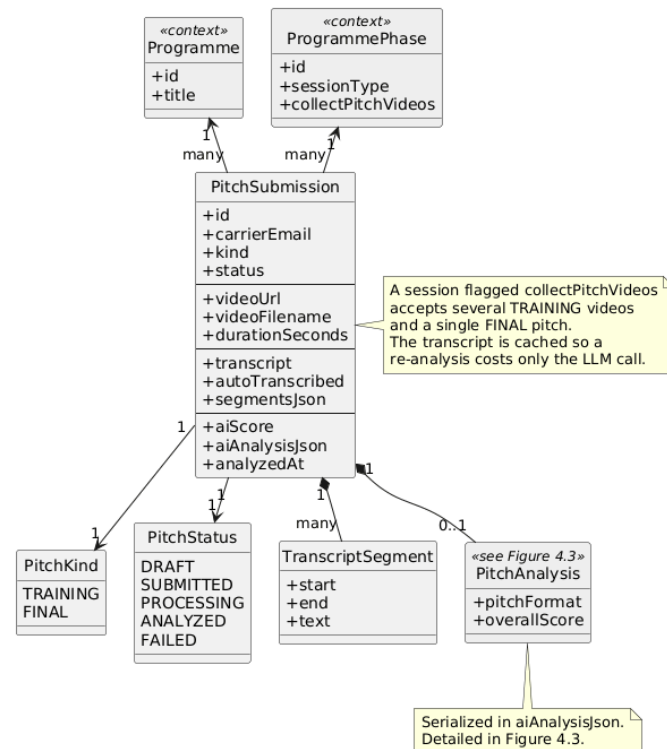


Figure 4.2 — Design class diagram: pitch submission

A submission is bound to a session flagged for video analysis and is typed by its **kind**: a carrier may deposit several TRAINING videos, to measure progress from one attempt to the next, but only one FINAL pitch. The transcript and its segments are stored on the submission so that re-running an analysis costs only the language-model call, the expensive speech-to-text step being reused.

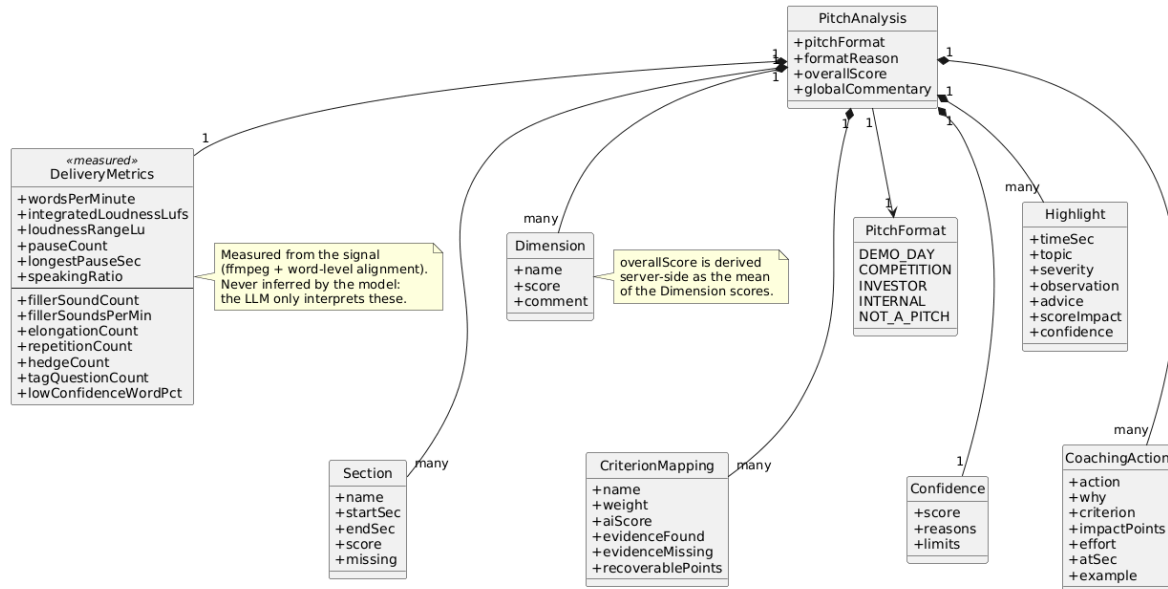


Figure 4.3 — Design class diagram: AI analysis result

Figure 4.3 makes the grounding principle of section 2.4.2 visible in the model. **DeliveryMetrics** is stereotyped <<measured>>: every one of its fields comes from the signal — ffmpeg for loudness and silences, word-level alignment for the hesitations — and none is produced by the language model, whose role is limited to interpreting them. The overall score is not a field the model fills in either: it is derived server-side as the mean of the Dimension scores, which keeps the grade reconcilable with the profile shown to the user.

4.1.2 Class Descriptions

Table 4.1 goes back over the role of each class in the domain.

Class	Description
User	A platform account ; carries one or more roles and fine-grained permissions ; backs carriers, juries, mentors, and administrators.
Organization / OrganizationMember	An applying entity and its team ; linked to applications.
Programme	An incubation programme: metadata, dates, sectors, criteria, sessions, and partners ; its status is synchronized from its sessions.
ProgrammeCriteria	A weighted evaluation criterion, used by juries and by Medi.

Class	Description
ProgrammePhase (Session)	The unified session entity (kept under the legacy table name `programme_phases`): typed, access-controlled, dated, possibly nested, with an optional activity agenda.
SessionDay / SessionActivity	The hourly agenda of a day-session.
Candidature	An application linked to a programme and an organization ; carries the custom answers, the consolidated score, its evaluations, and jury assignments.
Evaluation	The weighted score given to an application by a jury (or an administrator) for a given session (`candidature × jury × session`).
JuryAssignment	Assigns a jury (by email) to a session, with a tokenized, no-login evaluation link.
Task	An action item tied to a programme and assigned to a user.
Invitation	An archived communication (type, status, recipient, subject, token), used both for invitations and for tracked session notifications.

Table 4.1 — Description of domain classes

4.1.3 Data Dictionary

Programme

Attribute	Type	Description
id	Long	Unique identifier
title	String	Programme title
description	Text	Detailed description
type	enum	PUBLIC / PRIVATE
status	enum	DRAFT/OPEN/IN_PROGRESS/EVALUATION/CLOSED/CANCELLED/A
startDate / endDate	Date	Programme window
applicationDeadline	Date	Derived from the application session's end date
sectors	List	Targeted sectors

Table 4.2 — Data dictionary — Programme

ProgrammePhase (Session)

Attribute	Type	Description
id	Long	Unique identifier
title	String	Session title

Attribute	Type	Description
sessionType	enum	CANDIDATURE_SUBMISSION ... TRAINING_DAY
visibility	enum	VISIBLE / HIDDEN / PRIVATE
allowActivities	Boolean	False for application sessions
allowOverlap	Boolean	Allows overlapping within the same lane
durationKind	String	"day" or "range"
startDate / endDate	Date	Session window
status	enum	UPCOMING / ACTIVE / COMPLETED
lane	String	Swimlane / track
parentSessionId	Long	Parent range for a nested day

Table 4.3 — Data dictionary — ProgrammePhase (Session)

Candidature

Attribute	Type	Description
id	Long	Unique identifier
programmeId	Long	Target programme
organizationId	Long	Applying organization
carrierEmail	String	Carrier's email address
status	enum	PENDING ... ACCEPTED/REJECTED
totalScore	Double	Consolidated weighted score
customAnswers	JSON	Answers to the programme's own form

Table 4.4 — Data dictionary — Candidature

Evaluation

Attribute	Type	Description
id	Long	Unique identifier
juryId	Long	Evaluating jury
sessionId	Long	Evaluation session
weightedScore	Double	0–10, weighted by criteria
comment	Text	Free-form comment

Table 4.5 — Data dictionary — Evaluation

Invitation

Attribute	Type	Description
id	Long	Unique identifier
type	enum	JURY / PORTEUR / MEMBER / ORGANISATEUR / GUEST / GENERAL
status	enum	PENDING / SENT / FAILED / ACCEPTED / DECLINED
recipientEmail	String	Recipient
subject	String	Email subject
token	String	Unique tracking token
sentAt	DateTime	Delivery timestamp

Table 4.6 — Data dictionary — Invitation

4.2 Design Sequence Diagrams

4.2.1 Tracked Session Notification

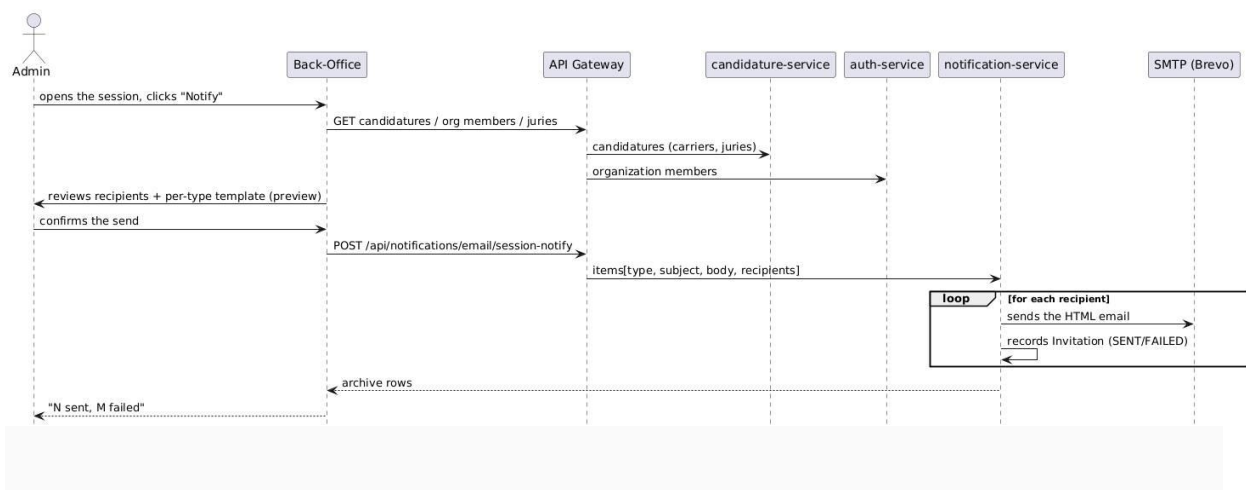


Figure 4.4 — Design sequence diagram: tracked session notification

4.2.2 Visibility-Based Session Filtering

This last sequence is worth dwelling on for a moment, since it's what actually enforces visibility server-side — a point a good deal of the platform's security rests on.

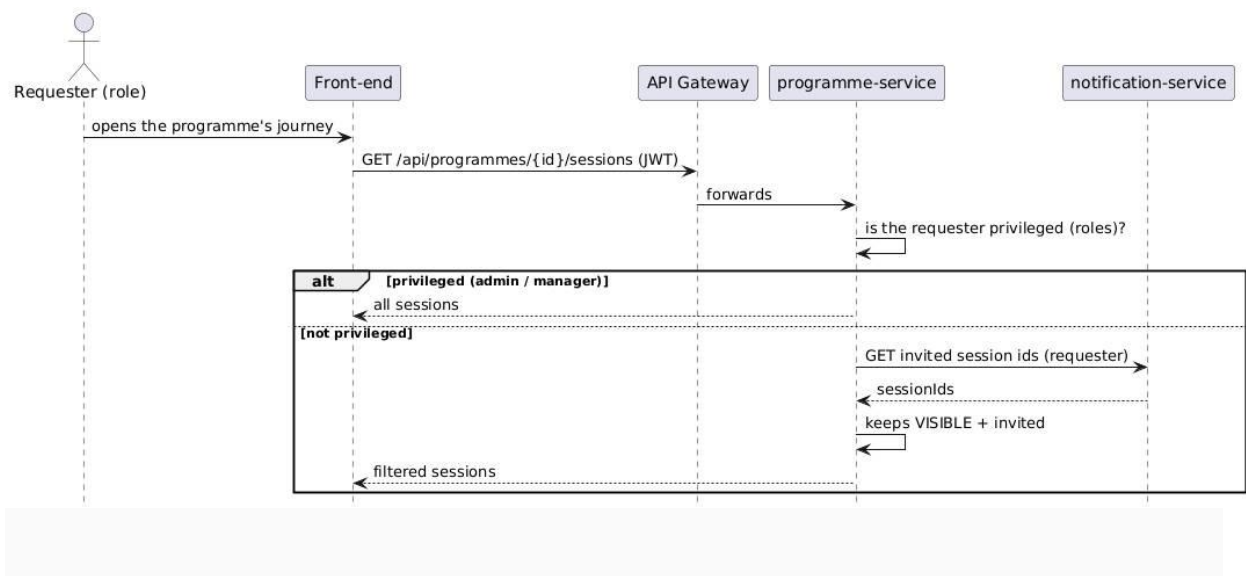


Figure 4.5 — Design sequence diagram: visibility-based session filtering

4.3 Component Diagram

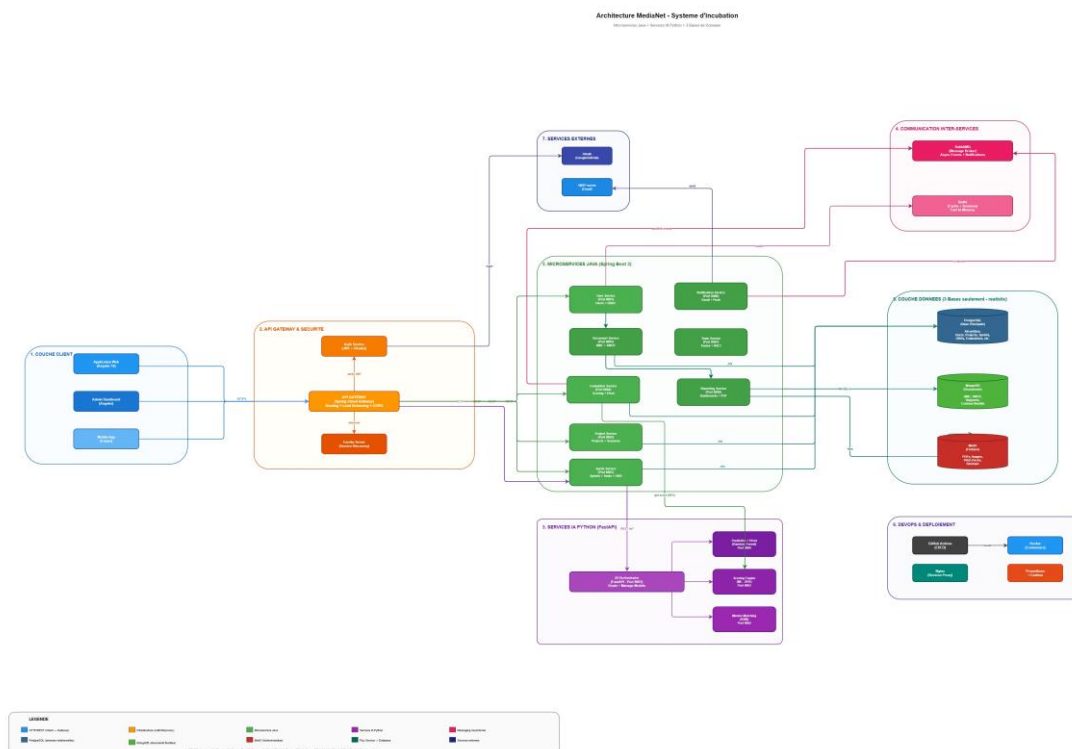


Figure 4.6 — Component diagram of the platform

Conclusion

What comes out of this is a **domain-driven, microservices** decomposition: an extensible session model sitting at the core, evaluation handled per session, access control that mixes

visibility with roles and invitations, and a communication channel that keeps a record of everything it sends. Between the class diagram, the data dictionary, the sequence diagrams, and the component diagram, there's enough here to actually start building. That's Chapter 5.

Chapter 5 — Implementation

Introduction

Design is the plan; this last chapter is what actually got built. It covers the work environment (software and hardware), the frameworks and languages used, the architecture adopted, the interfaces delivered, the test dataset, the planning that was followed, and the difficulties that came up along the way — together with what changed as a result.

5.1 Work Environment

5.1.1 Software Environment

Category	Tools
Operating system	Windows 11 (development), Linux containers (runtime)
Containerization	Docker, Docker Compose
IDEs	IntelliJ IDEA, Visual Studio Code
API & DB tools	Postman, pgAdmin / psql
Version control	Git
Object storage	MinIO (S3-compatible)
Transactional email	Brevo SMTP relay
Artificial intelligence	OpenRouter / local Ollama (LLM)

Table 5.1 — Software environment

5.1.2 Hardware Environment

Item	Specification
Development machine	[CPU], [RAM $\geq$ 16 GB recommended], [SSD]
Runtime	A Docker host running about a dozen containers (five services + five databases + gateway + two front-ends + MinIO)

Table 5.2 — Hardware environment

5.1.3 Frameworks

- **Back-end:** Spring Boot, Spring Web, Spring Data JPA, Spring Security, Spring Cloud Gateway, Spring Cloud Netflix Eureka, Hibernate, Lombok.
- **Front-end:** Next.js (App Router), React, Tailwind CSS, Framer Motion, Recharts, lucide-react, react-hot-toast, Axios.

5.1.4 Programming Languages

- **Java 17** — the microservices.
- **TypeScript / JavaScript** — the Next.js front-ends.
- **SQL** — PostgreSQL.
- **HTML / CSS** — markup and Tailwind styling.

5.2 Architecture Adopted

The platform rests on a **microservices** architecture, sitting behind an **API Gateway** with **service discovery**. Each service owns its own database and only exposes its REST API through the gateway — as shown in Figure 4.6.

Here's how responsibilities are split across the different services.

Service	Port	Responsibility
API Gateway	8080	Single entry point ; routing of <code>/api/**</code> ; CORS ; JWT forwarding
Eureka	8761	Service registry / discovery
auth-service	8081	Users, roles/permissions, organizations & members, JWT issuance, file uploads (MinIO)
programme-service	8086	Programmes, sessions (Journey), criteria, partners, tasks, catalogs ; validation layer ; visibility filtering
candidature-service	8083	Applications, jury assignments, evaluations, shortlists
notification-service	8087	Invitations, contacts/groups, session notifications, email archive
admin-ai-service	8088	AI scoring (Medi) and admin assistant

Table 5.3 — Service and port map

Security. Authentication rests on **JWT** and works in a **stateless** manner: the authentication service issues a signed token that embeds the user's identity, roles, and permissions. Each service then runs its own JWT filter, which validates that token and populates the security context ; controllers rely on it to authorize requests via `hasRole(...)` or `hasAuthority("module:action")`. Some sensitive reads — the list of sessions, in particular — go a step further: they're **filtered by role and by invitation**, which actually enforces visibility rules server-side, and not just in the interface.

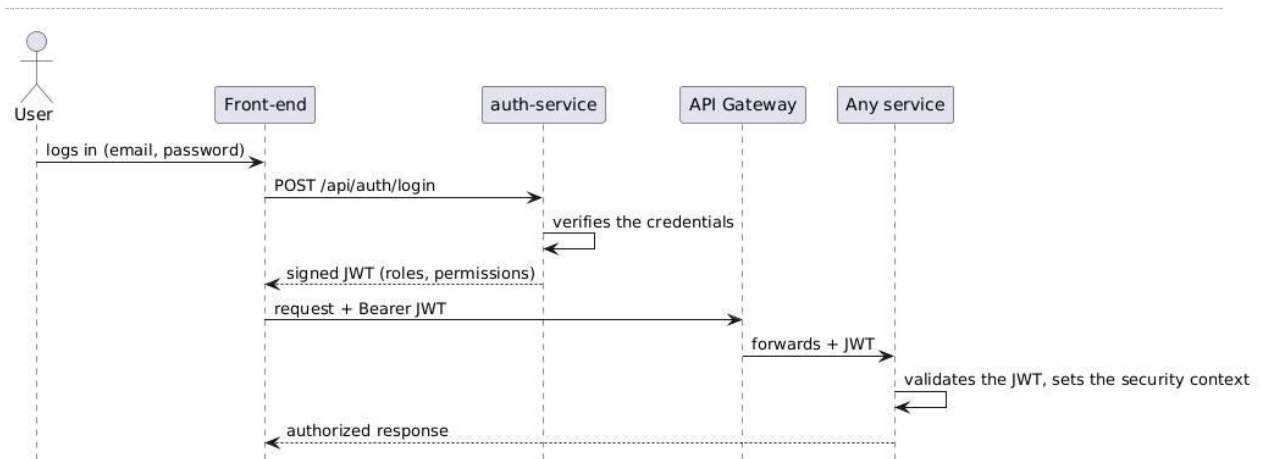


Figure 5.1 — Sequence diagram: stateless JWT authentication

5.3 Interfaces Delivered

This section walks through the main screens of the two front-ends ; each placeholder below should be replaced with the corresponding screenshot.

[Insert a screenshot of the login page]

Figure 5.2 — Login screen

[Insert a screenshot of the programme dashboard]

Figure 5.3 — Programme dashboard (back-office)

The programme dashboard acts as the back-office's central hub: it surfaces the key indicators (applications, evaluations, accepted), the upcoming sessions to notify, the list of missing items, and the top candidates.

[Insert a screenshot of the journey builder]

Figure 5.4 — Visual Journey builder (back-office)

The journey builder, meanwhile, displays sessions on a timeline organized by lane — day and range shapes, nesting, and an hourly agenda for day-sessions.

[Insert a screenshot of the session overlay]

Figure 5.5 — Session detail (Details / Activities tabs)

A session's detail view now opens on a well-structured **Details** tab — type, dates, status, visibility, description, related counts — while **activities** and feature panels stay in separate tabs, so the page doesn't get cluttered.

[Insert a screenshot of the evaluation dashboard]

Figure 5.6 — Evaluation dashboard and candidature review (with AI scoring)

[Insert a screenshot of the Participants tab]

Figure 5.7 — Participants hub and invitation template (placeholders + tracking link)

[Insert a screenshot of the front-office programme page]

Figure 5.8 — Public programme page (front-office)

On the front-office side, the programme page brings together public information, sessions — with their **type badges**, always respecting **visibility** — as well as criteria and partners.

[Insert screenshots of the application form and the jury evaluation page]

Figure 5.9 — Application form and jury evaluation page (front-office)

5.4 Project Planning

The table below retraces the planning followed during the project, organized into successive increments.

Sprint	Period	Content
Sprint 0	[Week 1–2]	Getting up to speed on the subject, studying the existing landscape, setting up the environment (Docker, Git repo, microservices skeleton)
Sprint 1	[Week 3–4]	JWT authentication, user and role management
Sprint 2	[Week 5–7]	Programme model and visual Journey builder (day/range sessions, nesting)
Sprint 3	[Week 8–9]	Online applications and customizable form

Sprint	Period	Content
Sprint 4	[Week 10–12]	Jury evaluation, weighted criteria, per-session re-evaluation
Sprint 5	[Week 13–14]	AI scoring integration (Medi)
Sprint 6	[Week 15–16]	Session visibility (VISIBLE/HIDDEN/PRIVATE) and centralized validation layer
Sprint 7	[Week 17–18]	Participants, personalized invitations, and communication archive
Sprint 8	[Week 19–20]	End-to-end testing, fixes, report writing

Table 5.4 — Project planning (sprints)

[Insert a detailed Gantt chart of the planning]

Figure 5.10 — End-of-studies project planning (Gantt chart)

5.5 Difficulties Encountered and Changes Made

The implementation wasn't exactly smooth sailing ; a few difficulties led to adjustments in the initial goals, and a couple of them are worth describing in a bit more detail than the others.

- **A naming quirk that took a while to get used to.** The session entity is called ``ProgrammePhase``, and it still lives in a database table named ``programme_phases`` — a name carried over from an earlier version of the project, before sessions became the rich, typed, nested thing they are today. Reading "ProgrammePhase" in the code and mentally translating it to "Session" every time took longer to get used to than expected, and more than once led to a diagram, or a variable name, that no longer quite matched what the class had grown into.
- **Evolution of the session model.** The session model was originally designed around a fixed type per programme. But feedback along the way quickly showed that a single programme needed to contain sessions of different natures, nested, each with its own visibility. The model was therefore generalized into the unified ``ProgrammePhase`` type described above, extensible without touching the schema.
- **Non-admin roles with the wrong access.** Early on, permission checks were too coarse: a jury or porteur account could occasionally reach an endpoint it had no business reaching, or find itself blocked from one it actually needed. Tracking these down meant going back through the controllers one by one and replacing broad role checks with fine-grained, per-module permissions (``hasAuthority("module:action")``) — the scheme the platform still uses today.
- **A date-related block on type changes.** An issue surfaced when simply changing a session's **type** — with no date changes at all — was, incorrectly, triggering a full

revalidation of date and overlap constraints, blocking otherwise legitimate edits. The fix: only revalidate dates when the edit actually touches them.

- **Overlap of nested sessions.** Moving a range that contained nested days could fail whenever those days ended up outside the new window. Since then, nested days are moved along with the range — or clamped to it — once the user confirms.
- **Perceived latency on the Tasks page.** The initial load was, mistakenly, waiting on data that wasn't essential to displaying the list. It's now loaded lazily, on demand.
- **LLM integration.** Making the AI score reliable required putting a fallback mechanism in place — default values whenever the language model's response didn't come back well-formed — so the jury is never left blocked.

A protocol mismatch between two services. The Java service called the Python media service and received an opaque 422. The cause had nothing to do with the payload: Java's HTTP client negotiates HTTP/2 by default, while the Python server only speaks HTTP/1.1. Pinning the protocol version fixed it.

An upload limit that ignored the configuration. Videos far below the configured limit were rejected with "Maximum upload size exceeded". The configuration file was correct; the environment variables declared in docker-compose were silently overriding it. The limit was raised to 2 GB consistently across the gateway, the service and the front-end.

A speech-to-text model that deletes hesitations. Whisper is trained to produce clean prose and removes every "uh" and "um", which made the whole point of the analysis impossible. The first hypothesis — the voice-activity filter — was tested and proved wrong: both settings gave the same result. Priming the model with a deliberately disfluent context sample makes it transcribe verbatim, recovering 59 hesitations on the same audio while preserving the content.

Measurements that looked right and were not. The detector reported 70 drawn-out sounds, but the words involved were "the", "and", "to" — function words. Whisper charges a word for the silence that follows it, so a word before a pause looks stretched. Subtracting the silences measured on the raw audio left 17 genuine ones. A number displayed with the authority of an objective measurement is worse than no number at all.

A vision model that was not reproducible. The model reported "arms crossed" for a speaker whose hand was raised — verified by extracting the frame. The same frame, run five times, produced different answers: the model was sampling. Its temperature and seed were pinned, and single-frame observations are now treated as weak evidence that cannot, on its own, justify coaching advice.

A scoring scale that collapsed. Told the scoring was too lenient, the rubric was tightened with absolute caps ("if the team or the ask is missing, the score cannot exceed 4"). The correction went too far: a deliberately poor talk and a pitch-competition winner both scored 3/10, so the scale no longer discriminated. The caps were replaced by per-dimension deductions, the expected content is now calibrated to the detected pitch format, and the overall score is computed server-side as the mean of the dimensions. Measured on five real videos, the spread went from 1.4 to 5 points.

An external model provider as a failure mode. The generation buttons hung indefinitely. The code was not at fault: the configured model had stopped responding. Two defects made a transient outage look permanent — no client-side timeout, and a four-minute server-side timeout inherited from a very different use case. Small generations now fail fast with a clear message, and a lighter model answers in under a second.

None of these took a single afternoon to sort out, and the naming confusion in particular kept resurfacing at odd moments — usually right when explaining the model to someone else made the mismatch obvious again. But collectively, these episodes left the data model, the access control, and the validation layer noticeably more solid than where they started.

5.6 Test Dataset

To validate the platform, a representative dataset was put together:

- **Users and roles:** at least one account per role — Administrator, Program Manager, Carrier, Jury, Mentor.
- **Organizations:** several startups with their members and project carriers.
- **Programme:** one complete programme with a full journey — an application session, a preselection session, a pitch day, an onboarding, an incubation range with nested training day-sessions, and a demo day — covering every session type, both shapes (day/range), nesting, and all three visibility levels (Visible / Internal / Private).
- **Applications:** a cohort of candidatures linked to the organizations, with custom answers.
- **Evaluations:** jury assignments per session and weighted scores, plus AI (Medi) scores for comparison.
- **Communications:** tracked invitations and session notifications, with SENT/FAILED archive entries.

This dataset was used to test the main flows: date validation (ranges, overlaps, nesting), role-based visibility filtering, the evaluation flow, AI scoring, and the notification/archive flow.

The table below lists the test scenarios that were run.

#	Scenario	Expected Result
T1	Create a range whose nested day would fall outside its bounds	Explicit error, or the nested day is moved along with the range
T2	Two overlapping sessions in the same lane	SESSION_OVERLAP_DETECTED (unless overlap is allowed)
T3	A non-privileged user lists sessions	Only VISIBLE (and invited) sessions are returned
T4	Submit an application after the deadline	Submission refused
T5	A jury scores the same candidate in two evaluation sessions	Two independent evaluations are recorded

#	Scenario	Expected Result
T6	AI-score an application	Per-criterion notes + weighted score + recommendation
T7	Send session notifications	Per-type emails sent + archived (SENT/FAILED)
T8	Change a session's type	Change persisted and reflected as a badge on the front-office

Table 5.5 — Test scenarios

Conclusion

What came out of this phase is the architecture that had been designed, now actually running: independent, secure microservices, two front-ends built for their respective roles, everything containerized and reproducible through Docker Compose. The main interfaces cover a programme's full lifecycle, and the platform held up against a representative test dataset. Not without a few bumps along the way — but those, as it turned out, pushed the initial model in a better direction.

Chapter 6 — Validation and Results

Introduction

The previous chapters described what was built. This one asks whether it works, and answers with measurements rather than with assertions. The question matters more here than in an ordinary application: a platform that grades a founder's pitch is trusted, and a tool that is trusted while being wrong is worse than no tool at all. This chapter therefore sets out the validation protocol, the results obtained on measurement reliability, score calibration and performance, and the functional test scenarios.

6.1 Validation protocol

Validating an evaluation tool cannot be done on a single recording: a grade only has meaning relative to other grades. A reference corpus of five real videos was therefore assembled, deliberately spanning the whole quality spectrum, so that the expected ranking was known in advance and independently of the platform.

Reference video	Expected level	Purpose
YC Demo Day pitch (Goodybag)	Excellent	Upper anchor: strong, proven traction
Pitch competition winner	Good	Should rank clearly above a poor talk
Elevator pitch winner	Good	Very short format (57 s)
Deliberately poor student talk	Weak	Lower anchor
Parody ("How to sound smart")	Not a pitch	Detects the degenerate case

Table 6.1 — Reference video corpus used for validation

The criterion of success is not an absolute grade — there is no ground truth for "this pitch deserves 7/10" — but **discrimination**: the ranking produced must match the known ranking, and the spread between the best and the worst must be wide enough for a jury to act on. As Table 6.1 shows, this corpus is replayed after every change to the rubric, which makes it a regression test rather than a one-off experiment.

6.2 Reliability of the measurements

Since the whole evaluation is grounded in measurements, the measurements themselves were validated first. Three results are reported, each corresponding to a defect that was found and corrected.

Recovering hesitations. As predicted by section 2.1.2, the transcription initially contained no filled pause at all: zero on a recording that audibly contains many. The hypothesis that the voice-activity filter was responsible was tested and rejected — both settings produced strictly identical output. Priming the decoder with a disfluent context sample recovered 59 hesitations on the same audio, while the content was preserved (0.94 word-level similarity with the clean transcript, filler words excluded). The technique costs no additional time.

Eliminating false positives. The first prolongation detector reported 70 drawn-out sounds. Inspection of the detail showed the words involved were "the", "and", "to" — function words, not hesitations. The cause is the alignment property described in section 2.2.3: the silence that follows a word is charged to that word. Subtracting the silences measured on the raw audio left 17 genuine prolongations, a 76 % reduction. The naive figure would have been displayed to the user as an objective measurement.

Reproducibility of the visual observation. The vision model described a speaker as having his arms crossed at 2:17; extracting the frame showed his hand raised. Run five times on that identical frame, the model produced different descriptions. Fixing the temperature and the random seed made the output reproducible and correct on three consecutive runs. This result is what justifies the design rule of Chapter 4: a visual observation is weak evidence.

Measurement	Before correction	After correction
Hesitations detected (same audio)	0	59
Prolongations detected	70 (function words)	17 (genuine)
Vision: same frame, 5 runs	inconsistent	identical and correct

Table 6.2 — Effect of the corrections on measurement reliability

6.3 Calibration of the scoring scale

The first rubric was reported as too lenient. The correction applied — absolute caps of the form "if the team or the funding request is missing, the score cannot exceed 4" — produced a worse failure, and the reference corpus revealed it immediately: **a deliberately poor talk and a pitch-competition winner both scored 3/10**. The caps applied before the strengths were read, so a Demo Day pitch showing 300 corporate customers and 93 % retention was capped for not presenting its team. A scale that cannot separate the best from the worst carries no information.

Three changes were made: a gap now lowers only the dimension it concerns instead of capping the overall score; the expectations are calibrated to the detected pitch format, so a two-minute Demo Day pitch is not penalised for omitting what that format never contains; and the overall score is derived server-side as the mean of the dimensions rather than chosen freely by the model. The corpus was then replayed.

Reference video	Format detected	Before	After
YC Demo Day (Goodybag)	Demo Day	4	7
Elevator pitch winner	Demo Day	4	5.9
Competition winner	Competition	3	5
Poor student talk	Not a pitch	3	2.3
Parody	Not a pitch	2.6	2
Spread (max – min)	—	1.4	5.0

Table 6.3 — Score calibration measured on the reference corpus

The result is unambiguous: the discriminating power went from **1.4 to 5.0 points out of 10**, the ranking now matches the expected one, and both winning pitches finally rank above the poor talk and the parody. Nothing about the videos changed — only the rubric. This is the central experimental result of the project, and it validates the calibration principle stated in section 2.4.2.

6.4 Performance

The first version of the pipeline processed a video in about 89 seconds, running transcription, audio analysis and vision one after another. As these three treatments are independent, they were parallelised, bringing the processing time down to about **13 seconds** — a factor of roughly seven. Caching the transcript removes the most expensive step entirely when an analysis is replayed.

A second performance result concerns the language model. The provider initially configured stopped responding, and every call expired. Benchmarking the models available on the same key showed a lightweight model answering in 0.5 s where the large one timed out, which settled the choice. Two defects were corrected at the same time: the absence of a client-side timeout, which made the interface wait indefinitely, and a four-minute server-side timeout inherited from a very different use case.

Indicator	Before	After
Video processing (5-minute pitch)	≈ 89 s	≈ 13 s
Replayed analysis (cached transcript)	full pipeline	LLM call only
Field generation (AI assistance)	timeout	≈ 0.5 – 8 s

Table 6.4 — Performance before and after optimisation

6.5 Functional test scenarios

Alongside the pipeline validation, the main functional paths were verified end to end on the deployed platform, each through the API gateway with a real account and a real token.

Scenario	Expected result	Status
Sign-in and JWT issuance	Token carrying roles and permissions	Passed
Live permission change (SSE)	Interface refreshed without reconnection	Passed
Programme creation as a draft, resumed later	Every step persisted and restored	Passed
Candidature session drives the closing date	Deadline derived automatically	Passed
Video upload (up to 2 GB)	Stored and linked to the submission	Passed
Full pitch analysis	Score, key moments and coaching returned	Passed

Private programme visibility	Listed only for invited users	Passed
------------------------------	-------------------------------	--------

Table 6.5 — Main functional test scenarios

Conclusion

This chapter answered the question it opened with. The measurements were validated one by one, and each validation caught a defect that would otherwise have reached the user: an absent hesitation count, an inflated prolongation count, an invented posture. The scoring scale, tested against a corpus whose ranking was known in advance, went from a state where it separated nothing to a spread of 5 points out of 10. Performance was divided by seven. **The essential result is methodological: every claim the platform makes about a pitch is now traceable to a measurement, and that measurement has itself been verified against its source.** The general conclusion draws the assessment of the project as a whole.

General Conclusion

Recap of the approach. The introduction made the problem clear from the start: incubation programmes, in practice, suffer from real fragmentation. Four stages followed from there. Studying the context and the existing landscape came first (Chapter 1). Then came a rigorous specification of functional and non-functional requirements, worked out through ten major use cases (Chapter 3). Next, an object-oriented design took shape around a unified session model and a per-session evaluation model (Chapter 4). And finally, all of it got built — a complete microservices platform, tested end to end (Chapter 5).

Presentation of results. Do the results answer the problem posed at the start? Largely, yes. **Medianet Incubateur** now brings together, in one secure tool, the visual design of a programme, online applications, AI-assisted jury evaluation, participant and organization management, and communications that can actually be traced back. Session **visibility**, enforced both server-side and in the interface, along with a **centralized validation layer** with explicit error codes, deliver the reliability and access control that existing solutions simply didn't have.

Problems encountered during implementation. Section 5.5 already goes into these in detail, so a short list will do here: a persistent naming mismatch between the 'ProgrammePhase' entity and the session concept it had grown into, permission checks that were too coarse for non-admin roles, the session model needing to be generalized later than expected, a validation bug that blocked legitimate session-type changes for a while, nested sessions needing special handling when a range moved, and the LLM integration needing a fallback to stay reliable. None of it required rethinking the architecture — each case was a targeted fix to the model, the access control, or the validation layer.

Contributions. Technically, this project meant working through the engineering of a genuinely non-trivial distributed system, start to finish: domain modeling and UML design, a microservices architecture built with Spring Cloud, stateless security with role-based access control, modern front-end work in Next.js and TypeScript, integrating an external LLM, DevOps with Docker Compose. Beyond the technical side, following an agile, iterative approach meant value got delivered progressively rather than all at once — and it forced a kind

of methodological discipline that's hard to learn any other way, on a project whose requirements kept shifting under it.

The contribution the project is most readily defended on, however, is methodological rather than purely technical: **it shows that an AI evaluation is only worth what its measurements are worth.** Three episodes documented in section 5.5 make the point concretely. A speech-to-text model that silently deleted every hesitation made the analysis meaningless until it was primed to transcribe verbatim. A detector reporting seventy drawn-out sounds was counting ordinary words followed by a pause, and only subtracting the measured silences brought it back to seventeen genuine ones. A vision model confidently described a speaker as having his arms crossed when the frame showed his hand raised — and, run five times on that same frame, it answered differently each time. In all three cases a plausible, confidently presented number was simply false. The calibration work that followed — deductions applied per dimension rather than as global caps, expectations calibrated to the detected pitch format, the overall score derived from the dimensions server-side — was validated on a reference set of real videos: the gap between the best and the worst pitch widened from 1.4 to 5 points out of 10, and a competition-winning pitch stopped scoring the same as a deliberately poor one. An evaluation tool that cannot tell those two apart is worse than none at all, because it is trusted.

Perspectives. A few things are worth pursuing further. Session types could become **fully catalog-driven** — migrating from a fixed enumeration to a referenced value, so future types (hackathons, investor days, ...) don't require touching the code at all. **Real-time** notifications (WebSocket / SSE) and calendar export (`.ics` / Outlook, alongside the Google Calendar link already in place) are another natural direction. So is strengthening **observability** — centralized logging, metrics, tracing — and building out a proper **CI/CD** pipeline with end-to-end tests. And the **analytics** side has room to grow: cohort funnels, inter-jury agreement, time-to-decision, maybe even a public, multilingual programme catalog. None of this was strictly required to answer the initial need. But it's exactly what a durable foundation should make possible.

Daily work and role within the team

Day to day, the work followed a regular rhythm: a short point with the supervisor on the priority of the moment, then development proper — reading the existing code before modifying it, implementing, testing locally against the full stack running under Docker, and deploying to the integration environment. A recurring part of each day went to diagnosis rather than to writing code: reproducing a defect, formulating a hypothesis, testing that hypothesis, and only then correcting. Feedback from the team, often given as a blunt observation — "it took too long", "the AI is too generous" — was systematically turned into a measurable problem before anything was changed.

The role held was that of a full-stack developer with real autonomy on the scope entrusted: the architectural choices of the media service, the design of the rubric and the evaluation protocol were proposed rather than received, then discussed and validated with the supervisor.

Skills acquired

Technically, the project consolidated back-end development (Java, Spring Boot, security, REST API design), front-end development (React, Next.js, TypeScript), relational modelling and its migration pitfalls, container-based deployment, and above all a field that was new: applied artificial intelligence — speech recognition, audio signal processing, vision models and the integration of language models, including the design and calibration of an evaluation rubric.

Methodologically, the skill that carried the most weight is the rigour of proof: never presenting a figure without having confronted it with its source. It was learnt the hard way, on measurements that were plausible and false. Alongside it came the ability to recognise one's own error — the over-correction of the rubric was a personal mistake before it was a technical one — and the habit of distinguishing a defect in one's own code from the failure of an external dependency, two situations that look identical from the user's side and are corrected in opposite ways.

Mutual contributions and fulfilment of the objectives

What the project brought to the company. Medianet has an operational platform covering the incubation cycle, and above all a differentiating module: an automatic pitch analysis that is entirely self-hosted, and therefore usable on founders' confidential videos without any third-party service being involved. The validation corpus and the protocol built around it remain usable by the team to re-qualify the rubric after any future change.

What the company brought to the project. Medianet provided a real subject rather than an exercise — with real users, real constraints and demanding feedback — the infrastructure needed to run local models, and a technical environment where architectural decisions had to be justified. The criticism received on the first version of the scoring is a good illustration: it is what led to the calibration work that became the scientific core of this project.

Were the objectives met? The functional objectives set at the start were met: the platform models a programme as a journey of typed sessions, manages applications and their evaluation, and enforces access control server-side. The objective attached to the second problématique — making the evaluation of a pitch objective, reproducible and useful — is met and measured: the

analysis is grounded in verified measurements, it is reproducible, it discriminates (5 points of spread against 1.4), and it never leaves the infrastructure. Two objectives remain partially open: the visual analysis of presence stays weak evidence, limited by the compact model used, and the evaluation of the coaching's real usefulness would require monitoring carriers over a full programme — which the duration of the project did not allow.

Bibliography / Webography

[1] Spring Boot Reference Documentation, Spring Team / VMware, online: <https://docs.spring.io/spring-boot/documentation.html> (accessed 2026).

[2] Spring Cloud Gateway Reference Documentation, Spring Team / VMware, online: <https://docs.spring.io/spring-cloud-gateway/reference/> (accessed 2026).

[3] Spring Cloud Netflix (Eureka) Reference Documentation, Spring Team / VMware, online: <https://docs.spring.io/spring-cloud-netflix/reference/> (accessed 2026).

[4] Next.js Documentation, Vercel Inc., online: <https://nextjs.org/docs> (accessed 2026).

[5] JSON Web Token (JWT) — RFC 7519, M. Jones, J. Bradley, N. Sakimura, IETF, May 2015, online: <https://www.rfc-editor.org/rfc/rfc7519> (accessed 2026).

[6] M. Fowler, J. Lewis, "Microservices: a definition of this new architectural term," martinofowler.com, 2014, online: <https://martinfowler.com/articles/microservices.html> (accessed 2026).

[7] OWASP Foundation, "OWASP Top Ten," online: <https://owasp.org/www-project-top-ten/> (accessed 2026).

[8] PostgreSQL Documentation, PostgreSQL Global Development Group, online: <https://www.postgresql.org/docs/> (accessed 2026).

[9] MinIO Documentation, MinIO Inc., online: <https://min.io/docs/minio/linux/index.html> (accessed 2026).

[10] Docker Compose Documentation, Docker Inc., online: <https://docs.docker.com/compose/> (accessed 2026).

To complete: add the specific academic references used during the project's literature review (papers on RBAC systems, AI-assisted evaluation, etc.).

Appendix A — Service and Port Map

Service	Port	Responsibility
api-gateway	8080	Entry point, routing, JWT forwarding
eureka-server	8761	Service discovery
auth-service	8081	Users, roles, organizations, JWT, files
candidature-service	8083	Applications, evaluations, jury assignments
programme-service	8086	Programmes, sessions, criteria, tasks, validation, visibility
notification-service	8087	Invitations, notifications, email archive
admin-ai-service	8088	AI scoring (Medi), admin assistant
pitch-media-service	8089	Video pipeline: transcription (Whisper), audio measurement (ffmpeg), vision
ollama	11434	Local model runtime (vision model for physical presence)
nextjs-frontoffice	3000	Public / carrier / jury front-end
nextjs-backoffice	3001	Administration front-end
PostgreSQL (×5)	5432 / 5434–5437	Database per service
MinIO	9000 / 9001	Object storage (S3-compatible)

The legacy `ai-scoring-service` (8084) and `ai-matching-service` (8085) have been superseded by Medi / admin-ai-service.

Appendix B — Main REST Endpoints

Method	Path	Description
POST	/api/auth/login	Authenticate and obtain a JWT
GET	/api/programmes	List public programmes
GET	/api/programmes/{id}	Programme detail (visibility-filtered phases)
GET	/api/programmes/{id}/sessions	List sessions (visibility-filtered)
POST	/api/programmes/{id}/sessions	Create a session
PUT	/api/programmes/{id}/sessions/{sid}	Update a session
POST	/api/candidatures	Submit an application
GET	/api/candidatures/programme/{id}	Applications for a programme
POST	/api/candidatures/{id}/assign-jury	Assign juries to a session
POST	/api/candidatures/{id}/evaluate	Submit a jury evaluation
POST	/api/admin-ai/score/{id}	AI-score an application
POST	/api/notifications/email/session-notify	Send tracked session notifications
GET	/api/notifications/invitations/programme/{id}	Communication archive
POST	/api/pitch/submissions	Create/update a pitch submission (training or final)
GET	/api/pitch/submissions/mine	The carrier's own pitch submissions
GET	/api/pitch/presentations/{programmeId}	Presentation sessions + submissions per session
PUT	/api/pitch/submissions/{id}/analysis	Persist the AI analysis on a submission
POST	/api/admin-ai/pitch/analyze/stream	Run the pitch analysis, streaming pipeline stages (SSE)
POST	/api/admin-ai/field-suggest	Generate or improve a single programme field
GET	/api/programmes/invited	Private programmes the caller was invited to

Method	Path	Description
GET	/api/notifications/invitations/my-programmes	Programme ids the caller was invited to

Table A.1 — Extract of main REST endpoints

Appendix C — Glossary

Term	Definition
Journey (Parcours)	The visual timeline of a programme's sessions and activities.
Session (Phase)	A typed stage of a programme (application, preselection, pitch day, ...).
Day / Range	A single-day session / a date-range session.
Carrier (Porteur)	A project carrier / startup founder who applies.
Jury	An evaluator who scores applications.
Visibility	Whether a session is Visible, Hidden (internal), or Private.
JWT	JSON Web Token, the stateless authentication credential.
RBAC	Role-Based Access Control.
Medi	The AI service providing LLM-based scoring.



ESPRIT SCHOOL OF ENGINEERING

www.esprit.tn - E-mail : contact@esprit.tn

Siège Social : 18 rue de l'Usine - Charguia II - 2035 - Tél. : +216 71 941 541 - Fax. : +216 71 941 889

Annexe : Z.I. Chotrana II - B.P. 160 - 2083 - Pôle Technologique - El Ghazala - Tél. : +216 70 685 685 - Fax. : +216 70 685 454